



上海大学
SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题 目：基于微服务架构的智能物流管理系统

学 院：计算机科学与工程学院

专 业：网络空间安全

学 号：21122119

学生姓名：付苗

指导教师：牛志华

起讫日期：2025 年 2 月 20 日起 6 月 2 日止



姓 名：付苗

学 号：21122119

论文题目：基于微服务架构的智能物流管理系统

原创性声明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签 名： 付苗 日 期： 2025 年 6 月

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

签 名： 付苗 指导教师签名： 日 期： 2025 年 6 月

摘 要

随着电子商务与跨区域配送需求的持续增长，传统物流管理系统在业务协同、资源调度与路径规划等方面逐渐暴露出响应迟缓、耦合度高和扩展困难等问题。为提升物流系统的可维护性、可扩展性与智能决策能力，本文设计并实现了一套基于微服务架构的智能物流管理系统。系统围绕订单履约、仓储库存、运输配送、路径规划与全国运力调度等核心业务展开，拆分为认证、用户、发货用户、商店与库存、订单、运输员、物流规划与网关等多个独立服务，基于 Spring Cloud Alibaba 生态完成服务治理，RabbitMQ 负责异步解耦，Redis 提供缓存加速。

在系统实现方面，后端采用 Spring Boot + Spring Cloud + MyBatis-Plus + Redis 技术栈；前端采用 Vue3 + Vite + Element Plus + Leaflet 构建多角色单页应用；部署采用 Docker Compose 一键编排。系统重点完成了四项内容：一是基于 BCrypt 与 JWT 的统一认证与网关鉴权机制；二是围绕订单、库存与配送的异步协同流程；三是集成 GraphHopper 路网引擎，实现基于 A* 的路径规划、基于 K-Means 的末端调度预览、基于最小费用最大流的全国运力规划以及大语言模型辅助决策；四是构建基于 Hub-and-Spoke 网络的完整履约流程。

系统创新点体现在三方面：以微服务架构支撑多角色物流协同闭环；将真实路网规划、聚类调度与运力优化算法引入统一平台；引入大语言模型形成“确定性算法求解 + 生成式模型裁决”的混合决策模式。

关键词：微服务架构；Spring Cloud Alibaba；路径规划算法；大语言模型；Hub-and-Spoke 物流网络

ABSTRACT

With the rapid growth of e-commerce and cross-regional delivery demand, traditional logistics systems increasingly suffer from poor scalability, tight coupling, and insufficient decision support. This thesis designs and implements an intelligent logistics management system based on microservice architecture, focusing on order fulfillment, inventory, transportation, route planning, and nationwide capacity scheduling. The system is decomposed into multiple independent services built on Spring Cloud Alibaba, with RabbitMQ for asynchronous decoupling and Redis for caching.

The backend adopts Spring Boot, Spring Cloud, MyBatis-Plus, and Redis; the frontend uses Vue3, Vite, Element Plus, and Leaflet; deployment uses Docker Compose. Key implementations include: unified authentication and gateway authorization based on BCrypt and JWT; asynchronous collaboration for orders, inventory, and delivery; integration of Graph-Hopper with A* route planning, K-Means dispatch preview, minimum-cost maximum-flow capacity planning, and LLM-assisted decision-making; and a complete Hub-and-Spoke fulfillment workflow.

The contributions are threefold: microservice architecture supporting multi-role logistics collaboration; integration of route planning, clustering dispatch, and capacity optimization into a unified platform; and a hybrid decision-making mode combining deterministic optimization with generative LLM reasoning.

Keywords: Microservice Architecture; Spring Cloud Alibaba; Route Planning Algorithm; Large Language Model; Hub-and-Spoke Logistics Network

目 录

摘 要 I

ABSTRACT II

1 绪论 1

 1.1 研究背景及意义 1

 1.2 国内外研究现状及不足 2

 1.2.1 国内研究现状 2

 1.2.2 国外研究现状 2

 1.2.3 现有不足与本文切入点 2

 1.3 本课题的研究内容 2

 1.4 论文组织结构安排 3

 1.5 本章小结 3

2 相关技术综述 5

 2.1 微服务架构相关技术 5

 2.1.1 注册中心 5

 2.1.2 微服务网关与服务调用 5

 2.1.3 消息队列 6

 2.2 数据持久化与缓存技术 6

 2.2.1 MySQL 与 MyBatis-Plus 6

 2.2.2 Redis 缓存 6

 2.3 前端技术栈 7

 2.4 容器化部署技术 7

 2.5 智能算法技术 7

 2.5.1 路网引擎与 A*搜索 7

 2.5.2 大语言模型辅助决策 7

 2.5.3 最小费用最大流 8

 2.6 本章小结 8

3 系统设计 10

3.1 需求分析	10
3.1.1 总体分析	10
3.1.2 功能需求-个人用户用例	11
3.1.3 功能需求-商业用户用例	12
3.1.4 功能需求-运输员用例	13
3.1.5 功能需求-仓库/网点用例	14
3.1.6 非功能需求	14
3.2 架构设计	15
3.3 系统详细设计	16
3.3.1 认证与权限模块设计	16
3.3.2 订单与库存模块设计	16
3.3.3 配送与路线模块设计	19
3.3.4 调度与运力规划模块设计	21
3.3.5 前端架构与页面设计	22
3.4 系统数据库设计	24
3.4.1 系统 E-R 模型	24
3.4.2 核心表设计	25
3.5 本章小结	26
4 系统实现	27
4.1 微服务环境搭建	27
4.2 认证与权限系统实现	27
4.3 核心业务服务实现	28
4.3.1 订单服务（order-service）	28
4.3.2 商店与库存服务（shop-service）	28
4.3.3 运输员配送服务（driver-service）	28
4.4 智能算法实现	28
4.4.1 GraphHopper 路网集成	29
4.4.2 A* 路径规划实现	29
4.4.3 LLM 路径决策实现（LlmJudgeRouteStrategy）	30
4.4.4 K-Means 聚类调度预览	30
4.4.5 MCMF 全国运力规划实现	31
4.4.6 Hub-and-Spoke 完整配送流程	32
4.5 前端实现	33

4.6 容器化部署实现	33
5 系统测试与运行结果分析	35
5.1 测试环境与评价指标	35
5.1.1 测试环境	35
5.1.2 评价指标	35
5.2 功能测试与业务链路验证	36
5.3 性能测试与结果分析	37
5.3.1 缓存优化效果验证	37
5.3.2 路径规划接口压力测试	37
5.3.3 并发下单与异步削峰验证	38
5.3.4 性能测试结论	39
5.4 多角色运行结果展示	39
5.4.1 发货用户端运行结果	39
5.4.2 商用户端运行结果	41
5.4.3 运输员端运行结果	42
5.4.4 管理员端运行结果	42
5.5 本章小结	43
结论	44
总结	44
特色创新	44
不足与展望	44
参考文献	46
附录 A 附录 A 核心接口清单	50
附录 B 附录 B 核心数据库表补充说明	51
附录 C 附录 C 关键算法参数与说明	52
致 谢	53

1 绪论

本章介绍论文的研究背景与意义、国内外研究现状及不足、主要研究内容以及论文组织结构。先讨论物流行业的发展现状与智能物流系统的现实需求，阐述微服务架构应用于物流领域的背景与意义；再从国内外两个视角梳理智能物流管理系统、路径规划与运力优化等方面的研究进展，分析现有工作的不足与本文的切入点；随后明确本文的研究内容；最后介绍论文的组织结构与各章安排。

1.1 研究背景及意义

物流行业作为支撑国民经济的基础性产业，在电子商务迅猛发展下其重要性日益凸显。然而，传统物流管理模式在业务复杂性、时效性与成本控制方面逐渐暴露出高成本、低效率、信息孤岛等瓶颈。中国的物流成本占 GDP 比重曾高达 14.1%，远高于发达国家平均水平。在物联网、大数据、人工智能等新一代技术驱动下，物流行业正经历数字化和智能化转型^[14]。例如，物联网技术使货物、车辆、仓库等物流要素的状态感知与数据采集成为可能^[2]；大数据技术为路径优化、需求预测提供决策支持^[3]；人工智能在智能调度、风险预警等方面的应用提升了物流系统的自动化水平^[4]。

与此同时，传统单体式系统架构在支撑复杂智能物流应用时局限性愈发明显：模块耦合度高、维护成本大、迭代周期长，难以满足高并发与高可用需求^[5]。微服务架构将复杂系统拆分为自治服务单元，天然契合智能物流系统对灵活性、可扩展性与高可用性的要求^[6]。基于此，本研究设计并实现基于微服务架构的智能物流管理系统，具有以下意义：

- (1) 提高物流运作效率：微服务架构解耦各业务模块，结合路径优化与智能调度，提升运输与仓储资源利用率。
- (2) 降低运营成本：弹性伸缩能力避免资源浪费，优化路径与库存管理降低运输与仓储成本。
- (3) 提升用户体验：实时订单跟踪与精准预计送达时间改善用户满意度，独立部署保障服务连续性。
- (4) 促进物流行业数字化转型：为传统物流企业向智能化转型提供可借鉴的技术方案。

1.2 国内外研究现状及不足

1.2.1 国内研究现状

在国内，物流信息系统经历了从单机管理软件到平台化云服务的发展过程。贾志刚等^[7]基于 Spring Cloud 微服务架构对物流运输管理系统进行了重构设计；王珊珊等^[8]基于 Spring Boot 实现了物流管理核心功能，但未涉及服务治理与多角色协同。在智能算法方面，李晓等^[9]将 K-Means 聚类算法应用于物流配送中心选址问题。根据中国物流与采购联合会数据^[10]，2023 年社会物流总费用占 GDP 比重仍高于发达国家水平。然而，国内现有工作多集中于单一功能模块优化，缺乏将路径规划、调度执行与运力规划统一的综合性研究，大语言模型在物流决策领域的工程化实践亦处于起步阶段。

1.2.2 国外研究现状

在国外，微服务架构在物流与供应链系统中的应用已有较多研究。Gonzalez 等^[11]分析了从传统 ERP 向微服务架构迁移的策略与挑战；Liu 等^[12]设计了基于微服务的智能物流管控平台。在路径规划领域，Zhang 等^[13]提出了改进的 A*算法用于车辆路径规划；Liu 等^[14]对 VRP 模型与算法进行了系统综述。在网络优化方面，Zhao 等^[15]构建了 Hub-and-Spoke 优化模型；Fragkos 等^[16]对最小费用最大流精确算法进行了综述。近年来，Simchi-Levi 等^[17]探讨了 LLM 在供应链决策中的潜力。

1.2.3 现有不足与本文切入点

综合国内外研究，现有工作存在以下不足：（1）多数系统仅关注单一业务环节，缺乏多角色全链路协同；（2）路径规划多基于理想化模型，较少在真实路网环境下裁决；（3）运力规划与调度缺乏与 Hub-and-Spoke 网络结合的工程实践；（4）大语言模型在物流领域的应用尚未形成“确定性算法 + 生成式模型”的混合决策模式。针对上述不足，本文以微服务架构为基础，设计并实现集成真实路网路径规划、K-Means 聚类调度、MCMF 运力规划与大语言模型辅助决策的智能物流管理系统。

1.3 本课题的研究内容

本课题的核心任务是设计并实现基于微服务架构的智能物流管理系统，主要研究内容包括以下五个方面：

- (1) 系统需求分析与架构设计：深入分析智能物流管理的核心业务流程和功能需求，设计基于微服务理念的系统总体架构，划分核心服务模块（如订单服务、库存服务、调度服务、路径优化服务等），并定义服务间的交互接口与协议。
- (2) 关键技术选型与应用：研究并选用合适的技术栈来实现微服务架构，重点围绕 Spring Cloud 生态进行技术实践，同时研究前端技术与后端微服务的交互模式。
- (3) 核心服务模块的设计与实现：针对订单管理、库存管理、运输调度、配送跟踪、路径优化等关键业务模块，进行详细的功能设计和数据库设计，并完成各微服务的编码实现。
- (4) 系统集成与部署：实现各微服务之间的集成与协同工作，并使用容器化技术对系统进行打包、部署和管理，以实现弹性伸缩和高可用性。
- (5) 系统测试与评估：对系统进行功能测试、性能测试和可用性测试，评估系统是否满足设计要求。

1.4 论文组织结构安排

全文共分为五章正文加结论与附录，整体结构围绕“需求分析—系统设计—系统实现—系统测试与运行结果分析—总结与展望”的逻辑展开。

第一章为绪论，介绍研究背景与意义、国内外研究现状及不足、主要研究内容以及论文整体结构。

第二章为相关技术综述，介绍系统所采用的微服务治理技术、数据访问与缓存技术、前端开发技术、容器化部署技术，以及路径规划、聚类调度、最小费用最大流和大语言模型等智能算法技术。

第三章为系统设计，围绕需求分析、总体架构设计、核心模块设计、前端设计与数据库设计展开。

第四章为系统实现，介绍微服务环境搭建、认证与权限系统实现、核心业务服务实现、智能算法实现、前端交互实现以及 Docker 容器化部署。

第五章为系统测试与运行结果分析，介绍测试环境与评价指标、功能测试与业务链路验证、性能测试与多角色运行结果展示。

1.5 本章小结

本章阐述了研究背景与意义，梳理了国内外研究进展与不足，明确了研究内容与论文结构，为后续章节奠定基础。

2 相关技术综述

本章围绕基于微服务架构的智能物流管理系统所采用的关键技术展开综述。内容主要包括微服务治理与服务通信技术、后端数据访问与缓存技术、前端开发与地图可视化技术、容器化部署技术，以及系统中涉及的路径规划、聚类调度、运力优化与大语言模型辅助决策等智能算法技术。这些技术共同构成了系统设计与实现的理论基础与工程支撑。

2.1 微服务架构相关技术

微服务架构将复杂系统拆分为围绕业务能力构建的自治服务单元，各单元可独立开发、部署与扩展^[6]。Zimmermann^[18]总结了微服务的核心特征，指出其在服务自治、独立部署与技术异构性方面的优势；Dragoni 等^[19]对其发展历程进行了回顾，认为演进方向正从架构风格向云原生与 DevOps 深度融合。相较于 SOA，微服务更强调细粒度拆分与独立运维^[20]，在成本效益方面亦有优势^[21]。

对于智能物流管理系统而言，其业务流程复杂、模块众多，且对可用性与可扩展性要求高。微服务架构能够将不同物流功能拆分为独立服务，便于快速迭代、按需扩展和故障隔离。

2.1.1 注册中心

微服务架构中，各服务实例的启停与扩缩容是常态，因此需要注册中心动态维护可用实例列表。主流方案的对比为：Zookeeper 采用 CP 模型保证强一致性，但网络分区时可能牺牲可用性；Eureka 采用 AP 模型优先保证可用性，但可能返回过期实例；Nacos 支持 CP 与 AP 模式自动切换，同时提供配置管理与服务发现能力。本系统选用 Nacos 作为注册中心。

2.1.2 微服务网关与服务调用

Spring Cloud Gateway 是 Spring Cloud 官方推出的 API 网关，基于 WebFlux 和 Reactor 模型构建，提供统一入口、请求路由、身份认证与权限控制等能力。Rodrigues 等^[22]对 API 网关的分类与挑战进行了梳理，指出网关需解决认证鉴权、限流熔断等横切关注点。OpenFeign 是声明式服务调用组件，配合 Spring Cloud LoadBalancer 可实现

基于服务名的实例发现与负载均衡。本系统使用 Gateway 作为统一入口实现 JWT 认证与路径权限控制，使用 OpenFeign 完成服务间同步调用。

2.1.3 消息队列

微服务架构中，服务间通信可分为同步调用与异步消息传递。消息队列提供服务解耦、异步通信与削峰填谷三方面能力。主流方案中，RabbitMQ 提供灵活路由与可靠消息确认，适合业务事件驱动场景；Kafka 以高吞吐和持久化著称，适合流处理场景^[23]。本系统在订单创建后的库存扣减、配送创建与调度入池等环节采用 RabbitMQ 实现异步解耦。

2.2 数据持久化与缓存技术

后端系统采用 MySQL 作为关系型数据库、MyBatis-Plus 作为数据访问增强框架，并结合 Redis 提升热点数据读取效率。

2.2.1 MySQL 与 MyBatis-Plus

MySQL 是广泛使用的开源关系型数据库，具备成熟的事务支持与索引机制^[24]，适合支撑订单、库存、配送等结构化业务数据存储。MyBatis-Plus 在保留原生 SQL 控制能力的基础上，提供通用 CRUD 接口、条件构造器与分页插件，可减少样板代码。各服务普遍采用 ServiceImpl<M,T> 构建业务服务层，借助 LambdaQueryWrapper 获得类型安全的条件查询。

相较于响应式数据访问方案（R2DBC^[25]、Spring WebFlux^[26,27]），本项目采用阻塞式 MyBatis-Plus + JDBC 路径。Kleppmann^[28] 指出，响应式方案在降低线程阻塞方面有理论优势，但编程模型复杂度显著高于传统阻塞式方案，在业务逻辑以同步编排为主的场景下收益有限。本系统核心链路属同步编排模式，阻塞式方案更符合工程复杂度要求。

2.2.2 Redis 缓存

Redis 是最主流的内存 KV 存储中间件^[29]，支持多种数据类型与持久化机制^[30]。本项目在多个服务中采用 Spring Cache 配合 Redis 实现 Cache-Aside 模式：读操作优先查缓存，未命中时查库并回填；写操作更新数据库后清除缓存。仓库列表、承运物详情、配送任务列表等高频读数据均采用该策略。认证方面采用无状态 JWT，Redis 不用于 Token 存储。

2.3 前端技术栈

前端采用 Vue3 + Vite 构建多角色单页应用。Vue3 的 Composition API 支持以函数为单位组织组件逻辑, 适合多角色应用中跨组件共享登录态与权限信息; Element Plus 提供统一的后台管理组件风格; Pinia 进行前端状态管理, 维护用户身份、认证令牌与角色标识等核心状态; Leaflet 实现运输轨迹与网络拓扑的地图可视化。前后端通过 axios 封装的统一 API 通信层交互, 通过请求拦截器注入 Authorization 头, 响应拦截器统一处理错误码与 Token 过期。

2.4 容器化部署技术

容器化部署将各微服务打包为独立容器镜像, 再利用编排配置统一管理, 实现弹性扩容与故障迁移。Balalaie 等^[31]的研究表明, 微服务与 DevOps 实践存在协同促进关系, 容器化编排是实现这一协同的关键基础设施。

Docker 是目前最主流的容器化平台^[32], 允许将应用及依赖打包为轻量级、可移植的容器镜像。结合持续交付实践^[33], 容器化消除了“环境差异”导致的部署风险。Docker Compose 通过 YAML 文件定义多容器应用的启动、停止与依赖关系, 本系统仅需一个全局配置文件即可完成数据库、中间件、微服务与前端的一键部署。对于毕设阶段的单机部署需求, Docker Compose 已足够; 生产环境可进一步引入 Kubernetes 获得自动扩缩容能力。

2.5 智能算法技术

本项目引入了面向物流优化场景的智能算法, 使系统从传统信息管理平台演进为具备辅助决策能力的智能物流平台。

2.5.1 路网引擎与 A*搜索

GraphHopper 是一款 Java 路径规划引擎, 可加载 OpenStreetMap 数据构建可查询的道路图结构, 基于真实道路拓扑执行路径搜索。A*算法通过代价函数 $f(n) = g(n) + h(n)$ 引导搜索, 启发函数采用 Haversine 球面距离, 满足可采纳性条件保证最优性。系统还使用加权启发策略, 通过调整 epsilon 生成多条差异化候选路线。

2.5.2 大语言模型辅助决策

大语言模型具备较强的自然语言理解与推理分析能力,在本系统中用于多个辅助决策场景:路径规划中结合候选路线与交通特征选择路线;末端调度中给出批次合并与优先级建议;运力规划中对 MCMF 结果进行可读分析。该方法不替代传统优化算法,而是与确定性算法协同提升决策的可解释性。

2.5.3 最小费用最大流

最小费用最大流 (MCMF) 问题求解在满足流量约束的前提下运输总成本最小的网络流分配。本系统将全国干线网络抽象为带容量与费用的有向图,通过 MCMF 求解生成跨区域运输的全局运力分配方案。

2.6 本章小结

本章梳理了系统实现涉及的关键技术并阐述了选型依据,核心决策汇总如下表。

表 2.1 核心技术选型对比与决策表

技术领域	选用方案		候选方案 1	候选方案 2	决策理由
注册中心	Nacos		Eureka	Zookeeper	同时支持服务发现与配置管理, CP/AP 可切换
消息队列	RabbitMQ		Kafka	ActiveMQ	路由灵活、消息确认可靠, 适合业务事件驱动
API 网关	Spring	Cloud Gateway	Zuul 1.x	-	基于 WebFlux 非阻塞模型, 与 Spring Cloud 2022 深度集成
ORM 框架	MyBatis-Plus		JPA/Hibernate	R2DBC	保留 SQL 控制能力, 通用 CRUD 减少样板代码
缓存方案	Redis	+ Cache-Aside	Caffeine 本地缓存	-	Redis 支持多实例共享, Cache-Aside 兼顾一致性与性能
前端框架	Vue3		React	Angular	Composition API 适合多角色状态复用, TypeScript 集成友好
地图组件	Leaflet		Mapbox GL	高德地图 JS API	开源免费, GeoJSON 渲染能力强, 无商业授权限制
容器编排	Docker	Com-pose	Kubernetes	-	毕设阶段单机部署足够, K8s 适用于生产环境

在微服务治理方面，系统基于 Nacos 实现服务注册与配置管理，基于 Spring Cloud Gateway 与 OpenFeign 实现统一接入与同步调用，基于 RabbitMQ 实现异步解耦；在数据访问方面，MySQL + MyBatis-Plus 提供结构化持久化，Redis + Cache-Aside 提供热点数据缓存；在前端构建方面，Vue3 + Element Plus + Leaflet 支撑多角色交互与地图可视化；在部署方面，Docker Compose 提供一键编排；在智能算法方面，GraphHopper、A*、K-Means、MCMF 与大语言模型构成了从路网加载、路径规划、聚类调度、运力优化到辅助决策的完整技术链。

3 系统设计

本章按需求分析→总体设计→详细设计→数据库设计的顺序展开。第一节分析系统涉及的角色用例、功能需求与非功能需求；第二节给出系统的总体架构设计与技术栈选型；第三节从认证权限、订单库存、配送路线、调度运力与前端架构五个方面展开详细设计；第四节独立给出系统数据库设计，包括全局 E-R 模型与核心表设计。

3.1 需求分析

3.1.1 总体分析

物流行业来自人类的工业运输与商业贸易需求，当今主要有两种表现形式：以合约委托的方式服务于企业；以计件付费的方式服务于个人。为了使我们的项目具备普遍意义，本系统分别考虑如下三种常见服务模式。

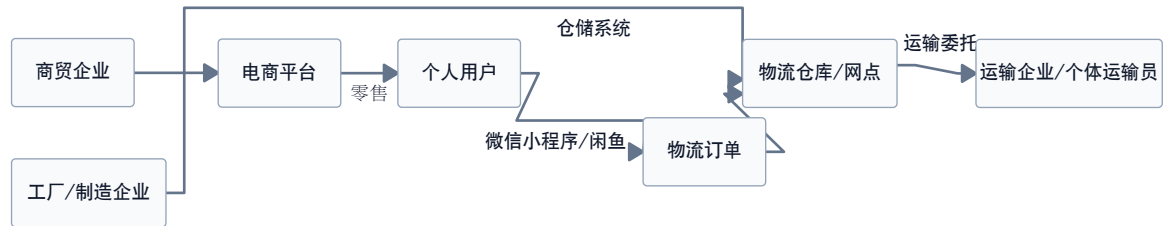


图 3.1 物流系统常见服务模式

如图所示，企业间通常会依据仓储系统的信息，将供应链的运输业务外包给物流系统。承运物到达商贸企业后，会通过淘宝、京东、拼多多等电商平台零售给个人。而个人也可能出于个人需求通过微信小程序等入口寄件，或针对闲鱼上的个人二手交易发起物流订单。物流公司通常会有分布在全国各地的仓库与服务网点，并将仓库/网点间的运输任务委托给运输企业或个体货车运输员。

综上所述，物流系统的参与方主要为四方：作为客户的工厂、商贸企业；作为客户的个人；作为平台的物流仓库、网点；作为第三方服务提供者的运输企业、货车运输员。可能和本系统交互的外部系统包括：仓储系统、电商平台、社交网络、支付系统。为了专注物流系统的本质，本项目略过支付系统，并为其他必要的外部接口仅提供最简单的实现。我们保留 API 接入功能和抽象层，只需通过 adaptor 模式进行简单的适配，即可接入这些外部系统。

以其中最复杂的物流订单模式为例,我们分析在此过程的流程及各个角色参与的步骤。

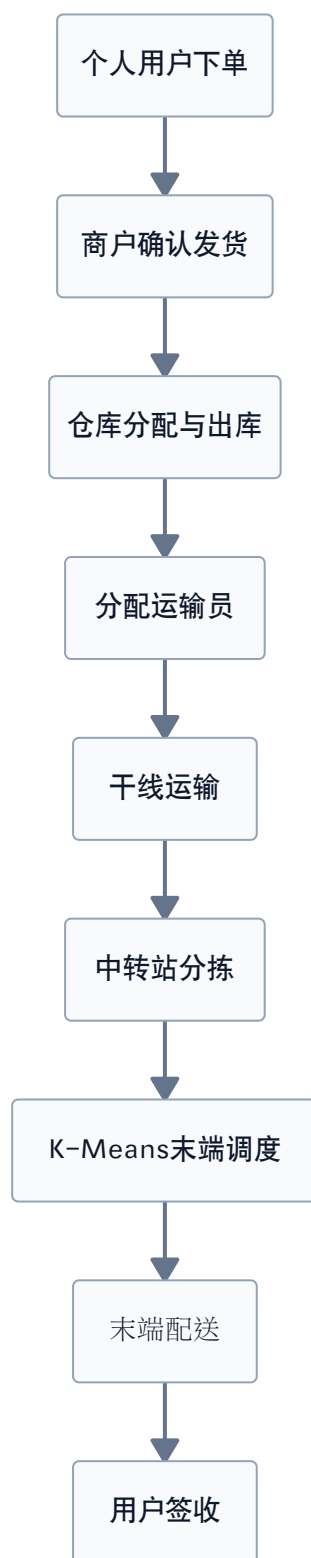


图 3.2 物流订单模式流程

3.1.2 功能需求-个人用户用例

个人用户主要是基于地址的物品收发以及在平台中发起物流订单。

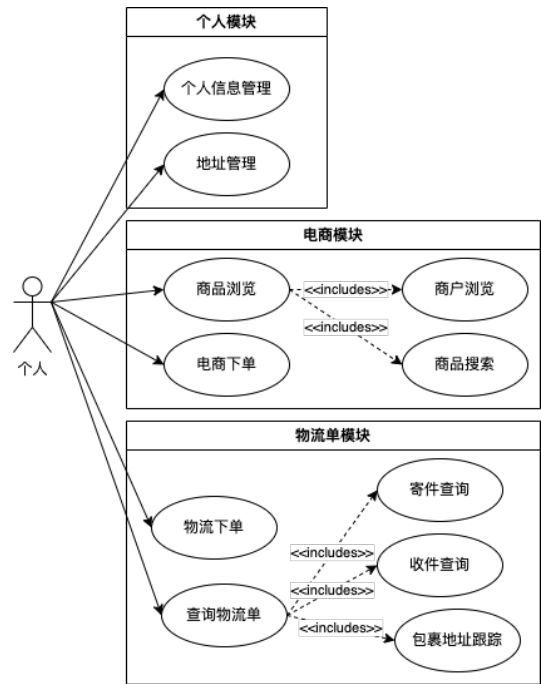


图 3.3 个人用户用例图

如图所示，个人用户需要地址管理、发起寄件、平台下单、物流下单、查看物流单等功能。

3.1.3 功能需求-商业用户用例

工厂/商贸的主要运输对象是仓库中的批量承运物。相比于个人,工厂/商户不使用地址,而是使用仓库作为地址。商户还可以在多个仓库间转运承运物,以优化库存管理和发货成本。智能物流系统应该能够帮助商户自动规划库存分配。

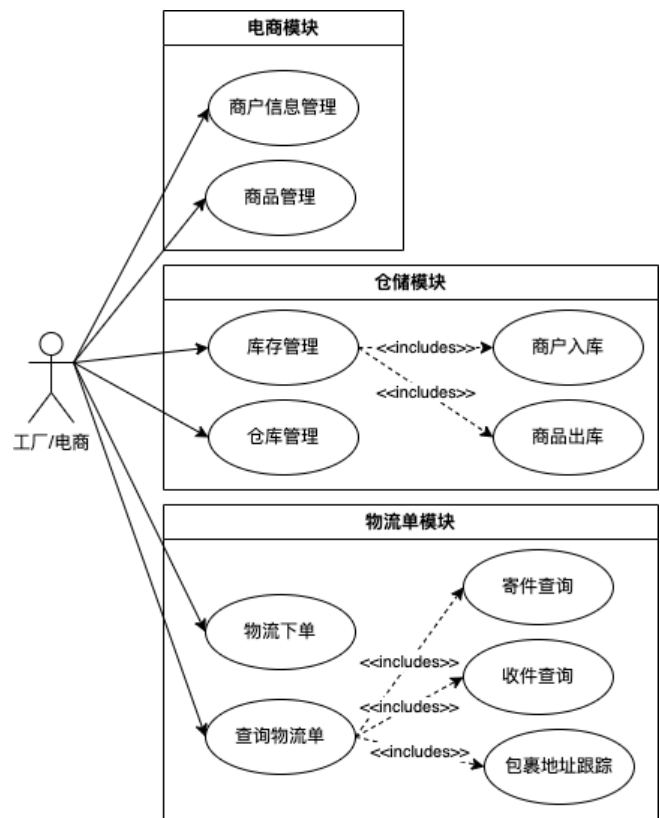


图 3.4 商业用户用例图

3.1.4 功能需求-运输员用例

运输员主要负责完成物流过程中的仓库到仓库的点到点运输，智能物流系统应该能基于路线、费用等问题自动规划出订单的最优安排，并为运输员自动规划最优运输路线。

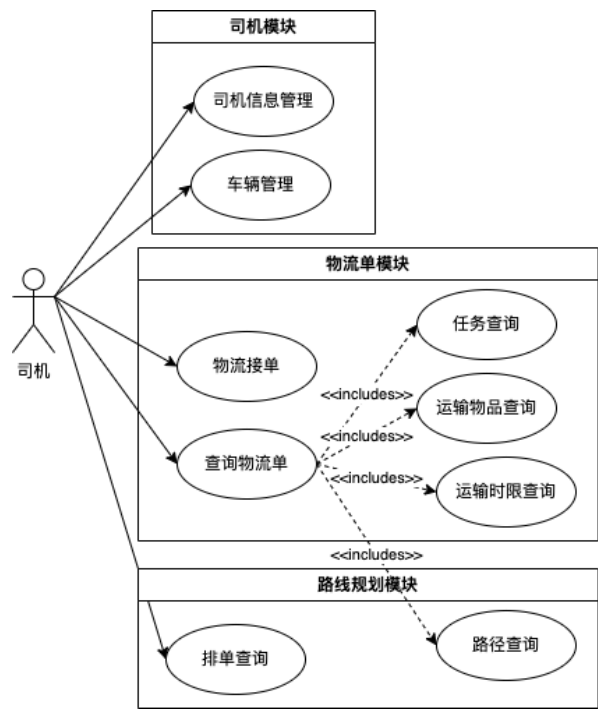


图 3.5 运输员用户用例图

3.1.5 功能需求-仓库/网点用例

仓库网点负责完成入库、出库、转运、派送等任务，这些任务可能通过物流公司配发的扫码设备采集包裹信息后自动触发至本系统中。

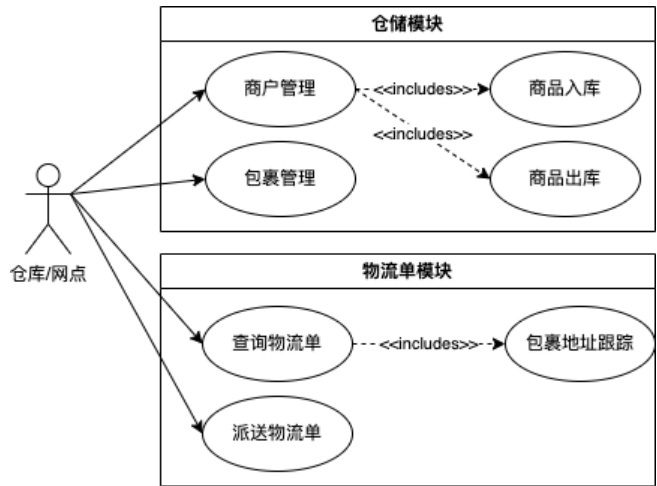


图 3.6 仓库/网点用例图

3.1.6 非功能需求

本项目致力于通过微服务架构解决现实世界中物流系统数据量大、使用环境复杂、崩溃后果严重等问题，因此需要解决如下非功能需求：

- (1) 高并发：该系统可借助横向扩展与弹性扩容等手段，合理应对高强度、随时变化的并发规模。在大型互联网架构实践中，高并发通常通过分而治之、缓

存加速与异步解耦三层策略实现^[34]，本系统的微服务拆分与 Redis 缓存正是遵循了这一思路。

- (2) 高可用：该系统可以容忍部分节点的故障、自动负载均衡并将任务迁移到可用节点。
- (3) 可扩展性：该系统可以轻松接入外部系统及数据，可以轻松实现新的功能以应对日新月异的功能需求。
- (4) 用户友好：该系统在 UI 设计上简洁易懂、易于使用。
- (5) 安全性：该系统为用户提供完善的鉴权机制与权限控制，避免恶意用户的风险操作。

3.2 架构设计

本系统采用基于微服务架构的分层设计思想，将复杂物流业务拆分为多个职责清晰的独立服务，并通过统一网关、服务注册发现、同步调用与异步消息机制实现协同^[35]。在服务拆分过程中，系统遵循领域驱动设计（DDD）原则^[36]，以业务能力边界划分服务职责，确保每个服务对应一个限界上下文。同时，基于 Conway 定律^[37]，系统的服务划分与团队协作结构保持对齐，以降低跨服务沟通成本。整体架构可划分为接入层、业务层、通信层、存储层和编排层五个部分。

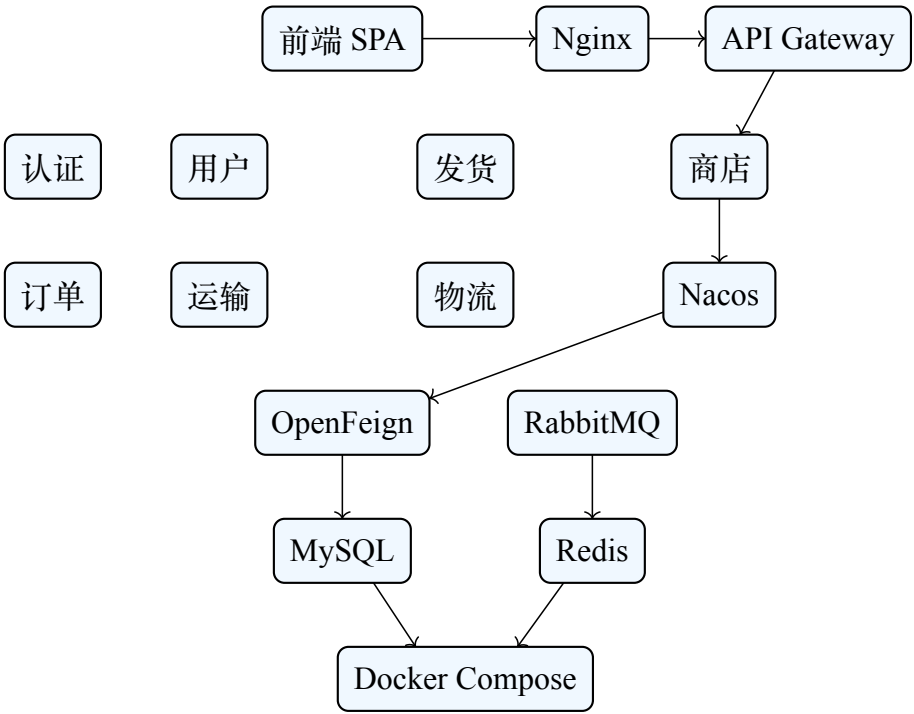


图 3.7 架构总览图

接入层由前端单页应用、Nginx 与 API 网关组成。业务层由认证服务、用户服务、发货用户服务、商店与库存服务、订单服务、运输员服务、物流服务及网关服务组成，各服务统一注册到 Nacos 中。通信层同时支持基于 OpenFeign 的同步服务调用和基于 RabbitMQ 的异步消息通信。存储层采用 MySQL 作为持久化数据库，并使用 Redis 作为缓存。编排层基于 Docker Compose 进行统一部署。

3.3 系统详细设计

基于前面的用例分析，系统详细设计围绕认证权限、订单库存、配送路线、调度运力与前端架构五个方面展开^[11]。以下依次阐述各模块的数据库设计与接口设计。

3.3.1 认证与权限模块设计

本模块主要负责用户管理、登录鉴权等基本功能，主要涉及用户实体类。

表 3.1 User 模型属性表

字段名	数据类型	约束	描述
id	BIGINT	PK AUTO_INCREMENT	用户 ID
username	VARCHAR(100)	NOT NULL, UNIQUE	用户名
secret	VARCHAR(255)	NOT NULL	BCrypt 密码哈希值
permission	TINYINT	NOT NULL	角色标识: 1=管理员, 2=发货用户, 3=商户, 4=运输员
create_time	DATETIME	NOT NULL	创建时间

表 3.2 系统管理模块 API 表

Method	URI	描述
POST	/api/auth/register	用户注册（发货用户直接通过；商户/运输员需审核）
POST	/api/auth/login	用户登录，返回 JWT Token
POST	/api/auth/logout	用户登出（前端清除本地 Token）
GET	/api/users/{userId}	获取用户信息
PUT	/api/users/{userId}/permission	管理员修改用户权限
DELETE	/api/users/{userId}	删除用户

3.3.2 订单与库存模块设计

本模块需要实现发货用户的个人信息与地址管理功能，以及商户的承运物管理、仓库管理与库存维护等功能。其中个人与地址间是一对多的关系；商户、承运物与仓库间涉及多对多关系，需通过关系表维护。以下分别给出核心实体与接口设计。

表 3.3 Person 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	发货用户 ID
user_id	BIGINT	关联 user.id
name	VARCHAR(100)	真实姓名
phone	VARCHAR(20)	联系电话
email	VARCHAR(100)	邮箱
create_time	DATETIME	创建时间

表 3.4 Address 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	地址 ID
customer_id	BIGINT	所属发货用户 ID
address	VARCHAR(255)	地址文本
is_default	TINYINT	是否默认地址

表 3.5 个人模块 API 表

Method	URI	描述
GET	/api/customer/{id}	获取个人信息
POST	/api/customer	添加个人信息
PUT	/api/customer	修改个人信息
GET	/api/customer/{id}/address	获取个人地址列表
POST	/api/customer/address	添加个人地址
PUT	/api/customer/address	修改个人地址
DELETE	/api/customer/address/{id}	删除个人地址

本模块同时涵盖物流业务中的商户、承运物、仓库与库存管理功能。其中涉及商户、承运物、仓库三个实体类。承运物与仓库间、商户与仓库间是多对多的关系，因此实现为承运物-仓库、商户-仓库关系表。而商户与承运物间是一对多的关系，无需关系表。

表 3.6 Shop 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	商户 ID
user_id	BIGINT	所属用户 ID
name	VARCHAR(100)	商户名称
phone	VARCHAR(20)	联系电话
status	TINYINT	审核状态：0=待审核，1=已通过

表 3.7 Product 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	承运物 ID
shop_id	BIGINT	所属商户 ID
name	VARCHAR(200)	承运物名称
price	DECIMAL(10,2)	申报价值
description	TEXT	备注说明
status	TINYINT	状态：0=待审核，1=已审核

表 3.8 Warehouse 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	仓库 ID
shop_id	BIGINT	所属商户 ID
name	VARCHAR(100)	仓库名称
address	VARCHAR(255)	仓库地址
latitude / longitude	DOUBLE	地理坐标

表 3.9 ProductWarehouse 库存关系表

字段名	数据类型	描述
id	BIGINT PK AUTO	库存 ID
product_id	BIGINT	承运物 ID
warehouse_id	BIGINT	仓库 ID
quantity	INT	库存数量

表 3.10 商户与承运物模块核心 API 表

Method	URI	描述
GET	/api/shop/{id}/products	获取商户承运物列表
POST	/api/shop/products	添加承运物

续表 3.10

Method	URI	描述
PUT	/api/shop/products	修改承运物
DELETE	/api/shop/products/{id}	删除承运物
GET	/api/shop/{id}/warehouses	获取商户仓库列表
POST	/api/shop/warehouses	添加仓库
GET	/api/shop/warehouses/inventory	查询库存
PUT	/api/shop/warehouses/inventory	设置库存

3.3.3 配送与路线模块设计

运输员模块负责运输员信息管理、车辆管理以及配送任务的接收与执行。其中运输员与车辆间是多对多关系，通过运输员-车辆关系表维护。配送任务是连接运输员与包裹的核心实体。

表 3.11 Driver 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	运输员 ID
user_id	BIGINT	所属用户 ID
name	VARCHAR(100)	运输员姓名
phone	VARCHAR(20)	联系电话
license_no	VARCHAR(50)	驾照编号
status	TINYINT	审核状态：0=待审核，1=已通过

表 3.12 Vehicle 模型属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	车辆 ID
plate_no	VARCHAR(20)	车牌号
vehicle_type	VARCHAR(50)	车辆类型
capacity	DECIMAL(10,2)	载重（吨）

表 3.13 运输员模块核心 API 表

Method	URI	描述
GET	/api/driver/{id}	获取运输员信息
POST	/api/driver	添加运输员信息
GET	/api/driver/{id}/vehicles	获取运输员车辆

续表 3.13

Method	URI	描述
POST	/api/driver/{id}/vehicles/{vid}	运输员绑定车辆
GET	/api/driver/{id}/tasks	查看配送任务

仓储模块负责仓库的日常管理，包括入库出库和包裹管理。仓库实体已在订单与库存模块中定义，此处不再重复。

表 3.14 仓储模块核心 API 表

Method	URI	描述
GET	/api/warehouse/{id}	获取仓库信息
GET	/api/warehouse/{id}/shops	获取托管商户
GET	/api/warehouse/{id}/products	获取仓库承运物
POST	/api/warehouse/{id}/stock/in	承运物入库
POST	/api/warehouse/{id}/stock/out	承运物出库

物流单模块负责订单主记录、物流路线以及运输状态的承载，是连接订单履约与配送执行的重要数据基础。

表 3.15 order_info 实际字段表

字段名	数据类型	描述
id	BIGINT PK AUTO	订单 ID
order_no	VARCHAR(32)	订单号（唯一）
customer_id	BIGINT	发货用户 ID
shop_id	BIGINT	商户 ID
address_id	BIGINT	收货地址 ID
total_amount	DECIMAL(10,2)	订单总金额
order_status	TINYINT	0=已取消, 1=待发货*, 2=待揽件, 3=运输中, 4=已送达（*初始状态）
payment_status	TINYINT	0=未支付, 1=已支付
warehouse_id	BIGINT	发货仓库 ID
origin_hub_id	BIGINT	起始中转站 ID
dest_hub_id	BIGINT	目标中转站 ID

表 3.16 delivery 配送单字段表

字段名	数据类型	描述
id	BIGINT PK AUTO	配送单 ID
order_id	BIGINT	关联订单 ID
driver_id	BIGINT	运输员 ID
delivery_type	TINYINT	0=普通配送, 1=末端配送
delivery_status	TINYINT	0=待承接, 1=已接单, 2=取件中, 3=配送中, 4=已送达
start_address	/ VARCHAR(255)	起终点地址
end_address		

表 3.17 订单模块核心 API 表

Method	URI	描述
POST	/api/order	创建订单
GET	/api/order/{id}	获取订单详情
GET	/api/order/list	获取订单列表
PUT	/api/order/{id}/status	更新订单状态

3.3.4 调度与运力规划模块设计

调度与运力规划模块是系统智能化能力的核心承载,负责路径规划、中转站管理、调度执行与全国运力规划^[15,16]。该模块涉及物流路线、中转站、运输连接、配送批次与调度池等多个数据实体,核心数据表定义如下。

表 3.18 logistics_route 核心字段表

字段名	数据类型	描述
id	BIGINT PK AUTO	路线 ID
route_no	VARCHAR(32)	路线号 (唯一)
segment_type	TINYINT	0=普通路线, 1=干线段, 2=末端段
route_status	TINYINT	-1=待激活, 0=待出发, 1=运输中, 2=已到达, 3=异常
planned_route	LONGTEXT	GeoJSON 规划路线数据

表 3.19 hub 枢纽属性表

字段名	数据类型	描述
id	BIGINT PK AUTO	枢纽 ID

续表 3.19

字段名	数据类型	描述
hub_name	VARCHAR(100)	枢纽名称
hub_level	TINYINT	0=全国枢纽, 1=省级中心, 2=城市配送中心
city	VARCHAR(50)	所在城市
daily_capacity	INT	日处理容量

表 3.20 logistics_batch 批次核心字段表

字段名	数据类型	描述
id	BIGINT PK AUTO	批次 ID
batch_no	VARCHAR(32)	批次编号（唯一）
batch_status	TINYINT	0=待发车, 1=干线运输中, 2=已到 Hub, 3=末端配送中, 4=已完成
total_orders	INT	批次订单数量

其余辅助表（hub_link 运输连接表、logistics_batch_item 批次项表、dispatch_pool 调度池表）的完整字段定义见附录 B。

表 3.21 物流规划模块核心接口表

Method	Path	描述
POST	/api/logistics/routes	创建物流路线并触发路径规划
POST	/api/logistics/routing/assign-hubs	分配起始与目标中转站
GET	/api/logistics/hubs	获取中转站列表
GET	/api/logistics/dispatch/pool	获取待调度订单池
GET	/api/logistics/dispatch/preview	K-Means + LLM 调度预览
POST	/api/logistics/dispatch/execute	执行调度并创建批次
POST	/api/logistics/dispatch/national/plan	触发全国运力规划
GET	/api/logistics/flow-plan/latest	获取最新运力规划详情
POST	/api/logistics/track/location	上报 GPS 坐标

3.3.5 前端架构与页面设计

本系统涉及四类角色，各角色交互模式存在显著差异，因此前端架构需解决多角色路由隔离、统一认证与地图可视化等问题。前端采用 Vue3 + Vite 构建单页应用，架构划分为视图组件层、路由控制层、状态管理层、通信服务层和部署接入层五个层次。

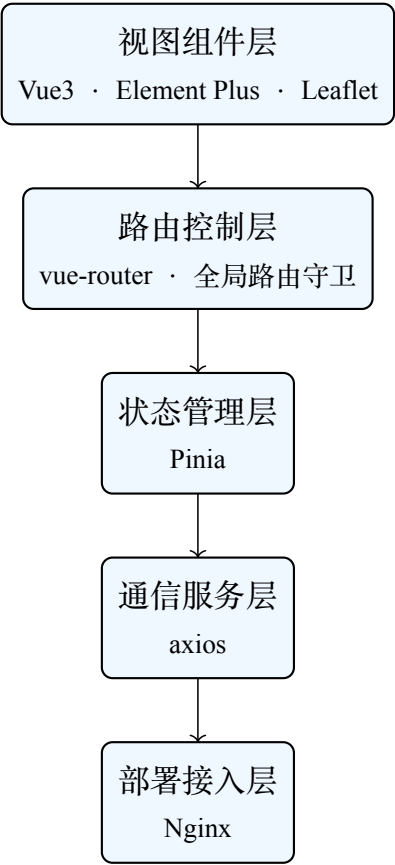


图 3.8 前端总体架构图

各层职责为：视图组件层由 Vue3 组件、Element Plus 与 Leaflet 构成；路由控制层由 vue-router 与全局路由守卫实现角色路由分发与访问控制；状态管理层采用 Pinia 维护用户身份、Token 与角色标识等核心状态；通信服务层基于 axios 封装统一 API 通信，自动注入认证头与错误码处理；部署接入层由 Nginx 托管构建产物，/api/请求反向代理至网关。

系统为四类角色设计了独立入口路径与权限边界，各角色页面结构如下表所示。

表 3.22 多角色页面—路由—功能对照表

角色	路由前缀	核心页面
管理员	/admin/	仪表盘概览、用户审核、中转站管理、末端调度预览、全国运力规划
发货用户	/customer/	发起寄件、订单查询、物流轨迹追踪
商户	/shop/	承运物管理、仓库管理、库存维护、待发货订单处理
运输员	/driver/	待处理任务列表、导航路线地图、配送状态更新、GPS 位置上报

各角色的路由入口由路由守卫统一控制，实现了登录校验、角色匹配与未授权拦截三个层面的访问控制。这种设计确保了角色间的数据隔离与操作安全，同时避免了各角色页面在权限判断上的重复实现。

3.3.5.1 关键交互设计

前端的关键交互围绕四个核心场景展开：登录鉴权流程中，JWT Token 持久化至 localStorage 并写入 Pinia，axios 拦截器自动附加认证头；订单流转交互中，前端调用订单创建接口后通过轮询获取履约进展，以时间线形式呈现各节点状态；运输员导航中，地图组件加载 GeoJSON 路线并结合 Leaflet 渲染，运输员端持续上报 GPS 坐标实现实时追踪；管理员调度预览中，K-Means 聚类结果以不同颜色渲染于地图，全国网络页面渲染 Hub-and-Spoke 拓扑与 MCMF 运力分配。

3.4 系统数据库设计

前面的详细设计中已展示了各模块的数据表定义，本节从全局视角对系统数据库进行统一梳理，首先给出系统 E-R 模型，然后对核心表进行集中说明。

3.4.1 系统 E-R 模型

系统核心实体按业务域可划分为用户与角色域、商品与库存域、订单与配送域、调度域以及网络与规划域五大部分，如图图 9 所示。在用户与角色域，User 表通过 permission 字段区分四类角色，Customer、Shop 与 Driver 分别通过外键与 User 形成一对一关联，实现角色公共属性与专属信息的分离；Customer 与 Address 为一对多关系。在商品与库存域，Shop 与 Product、Warehouse 分别为一对多关系，Product 与 Warehouse 通过 ProductWarehouse 关系表实现多对多关联，以支持同一承运物在多仓库中的分布式库存管理。在订单与配送域，OrderInfo 作为核心聚合实体，关联 Customer、Shop、Address 与 Warehouse；Delivery 与 LogisticsRoute 分别关联 OrderInfo 与 Driver，分别承载配送任务与路线规划数据。在调度域，LogisticsBatch 包含多个 BatchItem，每个 BatchItem 对应一个末端配送订单；DispatchPool 维护待调度订单，为 K-Means 聚类与批次执行提供输入。在网络与规划域，Hub 通过 HubLink 构成 M:N 自关联的干线网络拓扑，支撑 MCMF 运力规划；FlowPlan 与 FlowPlanItem 存储运力规划的全局结果与线路明细。

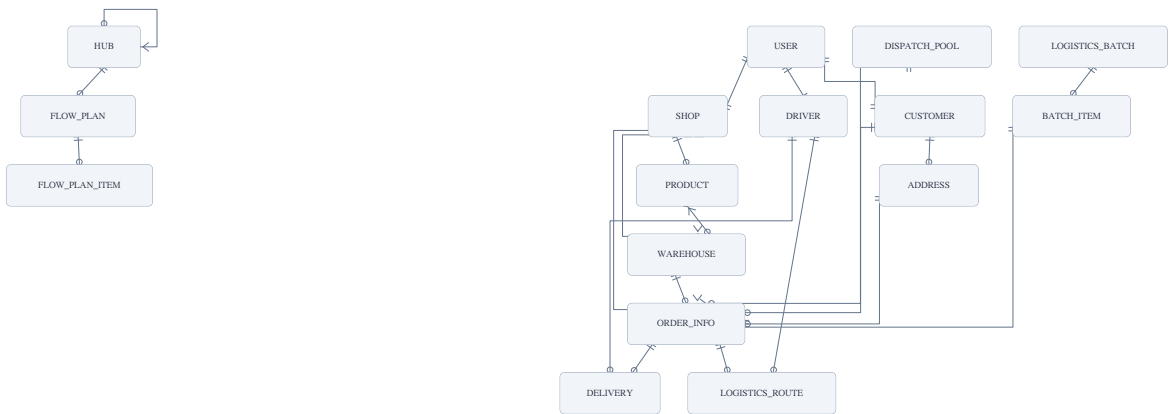


图 3.9 系统核心实体 E-R 图

为便于精确查阅各实体间的基数约束，下表对核心实体关系进行集中汇总。

表 3.23 系统核心实体关系表

实体 A	关系	实体 B	说明
User	1:1	Customer	一个用户对应一个发货用户身份
User	1:1	Shop	一个用户对应一个商户身份
User	1:1	Driver	一个用户对应一个运输员身份
Customer	1:N	Address	一个发货用户拥有多个地址
Shop	1:N	Product	一个商户拥有多个承运物
Shop	1:N	Warehouse	一个商户拥有多个仓库
Product	M:N	Warehouse	承运物-仓库多对多（库存表）
OrderInfo	N:1	Customer	多个订单属同一发货用户
OrderInfo	N:1	Shop	订单关联商户
OrderInfo	N:1	Address	订单关联收货地址
OrderInfo	N:1	Warehouse	订单关联发货仓库
OrderInfo	1:N	Delivery	一个订单对应多个配送单
Delivery	N:1	Driver	多个配送单由同一运输员执行
LogisticsRoute	N:1	OrderInfo	路线关联订单
LogisticsRoute	N:1	Driver	路线关联运输员
LogisticsBatch	1:N	BatchItem	批次包含多个批次项
BatchItem	N:1	OrderInfo	批次项关联订单
Hub	M:N	Hub	枢纽间通过 HubLink 连接
DispatchPool	N:1	OrderInfo	调度池关联待调度订单
FlowPlan	1:N	FlowPlanItem	运力规划包含多条线路明细

3.4.2 核心表设计

系统核心表在前文各模块设计中已逐一给出字段定义，此处对跨模块关联最密切的核心表进行集中梳理：

- (1) **user**: 统一用户表，通过 **permission** 字段区分四类角色，是认证与权限控制的基础。
 - (2) **order_info**: 订单主表，关联发货用户、商户、地址、仓库与中转站，承载订单全生命周期状态流转。
 - (3) **delivery**: 配送单表，关联订单与运输员，区分普通配送与末端配送。
 - (4) **logistics_route**: 物流路线表，存储 GeoJSON 规划路线数据，区分普通/干线/末端三种段类型。
 - (5) **logistics_batch**: 批次表，将末端配送订单按聚类结果分组，记录停靠顺序与配送状态。
 - (6) **hub / hub_link**: 枢纽与运输连接表，构成全国干线网络拓扑，支撑 MCMF 运力规划。
 - (7) **dispatch_pool**: 调度池表，维护待调度订单，是调度预览与执行的输入数据源。
- 其余辅助表的详细字段定义见前文各模块设计小节或附录 B。

3.5 本章小结

本章从需求分析、总体设计、详细设计与数据库设计四个方面对系统进行了系统性说明。基于多角色业务需求明确了功能边界与非功能目标；围绕微服务架构给出了分层架构设计；从认证权限、订单库存、配送路线、调度运力与前端架构五个维度展开详细设计；从全局视角给出了 E-R 模型与核心表梳理。

4 系统实现

本章按认证鉴权→核心服务→智能算法→前端交互→容器化部署的顺序阐述各模块的实现策略与关键技术机制，重点阐述实现过程中的设计决策与技术难点，而非罗列代码细节。

4.1 微服务环境搭建

为了保证各微服务在开发、测试与部署阶段具备一致的运行环境，系统首先完成了统一的微服务开发环境搭建。后端采用 Spring Boot 3、Spring Cloud 2022 与 Spring Cloud Alibaba 构建微服务体系，前端采用 Vue3 + Vite 完成页面开发。在服务治理方面，系统使用 Nacos 作为注册中心，各服务统一配置注册地址并注册至公共命名空间。服务端口采用固定划分方式，网关与各业务服务各自占用独立端口，便于开发调试与容器编排时的端口映射。

在项目结构方面，系统采用 Maven 多模块结构组织各微服务，每个微服务拥有独立的 pom.xml 文件管理自身依赖，并通过父 POM 统一管理 Spring Boot 与 Spring Cloud 的版本号，避免各服务间的依赖冲突。数据库迁移采用 SQL 脚本文件方式管理，各服务的建表语句按服务维度独立维护在 database 目录下。

4.2 认证与权限系统实现

系统认证与权限控制由认证服务与网关共同完成，采用“认证服务签发 + 网关统一鉴权 + 下游透传消费”的三段式架构。注册时序为：前端提交注册请求 → auth-service 校验用户名唯一性 → BCrypt 加密密码 → 写入 user 表 → 若角色为商户/运输员则额外写入角色信息表（待审核状态）。登录时序为：前端提交登录请求 → BCrypt 校验密码 → 检查审核与禁用状态 → 生成 JWT（封装 userId、username、roleCode） → 返回 Token。鉴权时序为：前端携带 Token 发起请求 → 网关过滤器拦截 → 检查白名单 → 验证 JWT → 路径级权限判断 → 附加身份透传请求头 → 转发至下游服务。

该设计将“身份建立”与“访问控制”解耦至不同服务：注册与登录由认证服务独立处理，网关仅负责令牌校验与路径授权，下游服务通过请求头获取用户身份而无需重复解析 JWT。系统根据角色类型实施差异化审核策略：普通发货用户注册后可直接使

用；商户与运输员需经管理员审核。路径权限规则与角色一一对应：`/api/customer/`仅允许发货用户、`/api/shop/`与`/api/stock/`仅允许商户、`/api/driver/`仅允许运输员、`/api/logistics/dispatch/`仅允许管理员，形成了“白名单 + 令牌校验 + 路径授权”的组合机制。

4.3 核心业务服务实现

4.3.1 订单服务（order-service）

订单服务是业务编排最复杂的服务，采用“同步校验 + 异步扣减 + 最终一致性”策略。首先通过 Feign 同步校验收货地址归属与承运物状态，在多个仓库中寻找库存充足的发货仓，创建订单主记录（初始状态 `order_status=1` 待发货，`payment_status=1` 已支付）。订单落库后，通过 RabbitMQ 发送库存扣减消息异步执行实际扣减，若失败则进入死信队列待补偿；同时发送配送创建消息与调度入池消息，完成下单链路的闭环。

在下单链路中，系统需要解决的核心挑战是跨服务数据一致性。订单创建涉及发货用户服务（地址校验）、商店服务（承运物与库存查询）、物流服务（中转站分配）等多个服务的协同。系统将需要即时反馈的校验逻辑（地址归属、承运物状态、库存充足性）放在同步链路中完成，确保用户能立即获知下单是否成功；而将可以延迟执行的副作用（库存扣减、配送创建、调度入池）放入异步消息，由消费者保证最终一致性。这种设计使用户下单响应时间保持在 100-200ms 范围内，不因下游消费速度而阻塞。

4.3.2 商店与库存服务（shop-service）

商店与库存服务承担承运物管理、仓库管理、库存维护等职责。在仓库管理方面，系统集成了高德地图地理编码，新增或修改仓库时自动获取经纬度坐标；若调用失败则保留原有坐标而不中断主流程。在性能优化方面，仓库列表查询采用 Spring Cache + Redis 缓存机制，写操作及时清除缓存以保持一致性。

4.3.3 运输员配送服务（driver-service）

运输员配送服务围绕配送单状态机组织业务流程：待承接(0)→已接单(1)→取件中(2)→配送中(3)→已送达(4)。服务以消息消费机制承接两类配送任务：商户发货后的首段配送创建，以及干线到达中转站后的末端激活。当前版本中，全国干线配送任务由管理员统一调度，运输员端仅展示末端配送任务。服务对运输员首页的高频访问“进行中配送列表”进行了 Redis 缓存优化。

4.4 智能算法实现

4.4.1 GraphHopper 路网集成

系统在物流服务中集成了 GraphHopper 作为底层路网引擎。GraphHopper 可加载 OpenStreetMap 地图数据并构建可查询的道路图结构，首次启动时需要对 OSM 数据进行建图，后续重启时可直接加载缓存，避免重复建图带来的启动延迟。系统配置了 OSM 数据路径、图缓存路径及路由策略等关键参数，使物流服务能够基于真实道路拓扑执行路径搜索。

4.4.2 A* 路径规划实现

系统实现了基于 A* 算法的路径规划策略^[13]。A* 算法的核心思想是通过代价函数 $f(n) = g(n) + h(n)$ 引导搜索方向，其中 $g(n)$ 为起点到当前节点的实际代价， $h(n)$ 为当前节点到目标节点的启发式估计。启发函数采用 Haversine 球面距离，由于该距离不会高估真实道路距离，因此满足可采纳性条件，能够保证搜索结果的最优性。以下给出 A* 路径规划的核心伪代码：

算法 4.1 A* 路径规划算法

```
function AStarRoute(start, goal, epsilon):
    openSet ← {start}
    gScore[start] ← 0
    fScore[start] ← epsilon × h(start, goal)
    while openSet ≠ □:
        current ← argmin_{n ∈ openSet} fScore[n]
        if current = goal:
            return ReconstructPath(cameFrom, current)
        openSet ← openSet \ {current}
        for each neighbor of current:
            tentative_g ← gScore[current] + d(current, neighbor)
            if tentative_g < gScore[neighbor]:
                cameFrom[neighbor] ← current
                gScore[neighbor] ← tentative_g
                fScore[neighbor] ← tentative_g + epsilon × h(neighbor, goal)
            if neighbor ∉ openSet:
```

续算法 4.1

```
openSet  $\leftarrow$  openSet  $\cup$  {neighbor}  
  
return failure
```

在标准 A* 基础上，系统还实现了 Weighted A* 机制，通过设置 $\epsilon=1.0$ 、1.8 与 3.5 生成多条具有风格差异的候选路径。 $\epsilon > 1$ 时算法倾向于更激进地探索靠近目标的节点，虽然不保证最优解，但能在更短时间内生成与标准 A* 路线不同的替代方案，为后续 LLM 裁决提供多样化的候选输入。

4.4.3 LLM 路径决策实现 (LlmJudgeRouteStrategy)

系统实现了大语言模型辅助的路线裁决策略，利用大语言模型在多候选路线之间进行综合判断。该策略的设计动机在于：A* 算法虽然能生成多条候选路线，但仅基于距离和道路拓扑进行评价，无法考虑历史交通特征、时段拥堵规律等上下文信息；而大语言模型具备对多维度信息进行综合分析的能力，适合作为路线选择的“二次裁决”环节。具体流程如下：

- (1) **候选路线生成**：分别以 $\epsilon=1.0$ 、1.8、3.5 调用 A* 算法，生成三条具有风格差异的候选路线（标准最短路线、次优绕行路线、远距高速路线）。
- (2) **上下文信息组装**：查询订单对应的出发时段、历史交通特征，将三条路线的距离、预计时间、途经道路类型等信息与上下文合并为结构化提示词。
- (3) **模型调用与结果解析**：调用 DeepSeek API，要求模型以 JSON 格式返回推荐路线编号与推荐理由。系统解析返回结果并选择对应路线。
- (4) **降级容错**：若 LLM 调用超时或返回格式异常，系统自动降级返回 $\epsilon=1.0$ 的标准 A* 最优路线，确保路径规划功能始终可用。

4.4.4 K-Means 聚类调度预览

系统提供了基于 K-Means 的调度预览能力^[9]。该功能的设计目标是为管理员提供末端配送方案的可视化预览与决策辅助，在实际执行调度前即可评估聚类效果与配送路线合理性。实现过程中，系统首先从调度池获取待调度订单，以收货地址的经纬度坐标为特征向量，执行 K-Means 算法进行空间聚类。K 值由管理员根据当前待调度订单量与可用运输员数指定，距离度量采用 Haversine 球面距离以匹配地理空间的实际距离关系。以下给出 K-Means 聚类调度的核心伪代码：

算法 4.2 K-Means 末端聚类调度算法

```

function KMeansDispatch(orders, k, maxIter):
    centroids ← SelectInitialCentroids(orders, k)
    for iter ← 1 to maxIter:
        clusters ← □
        for each order in orders:
            c ← argmin_{j ∈ 1..k} Haversine(order.location, centroids[j])
            clusters[c] ← clusters[c] ∪ {order}
        newCentroids ← □
        for j ← 1 to k:
            newCentroids[j] ← MeanLocation(clusters[j])
        if newCentroids ≈ centroids:
            break
        centroids ← newCentroids
    for each cluster in clusters:
        cluster.stops ← GreedyTSP(cluster.orders)
    return clusters

```

算法初始化阶段采用 K-Means++ 策略选择初始聚类中心，以减少随机初始化对结果稳定性的影响；收敛条件为聚类中心偏移量小于阈值（ 0.001° ）或达到最大迭代次数（默认 100 次）。聚类完成后，系统对每个簇内部执行近似 TSP 贪心排序，以优化配送停靠顺序、减少往返距离。系统还可选地调用大语言模型对聚类结果进行二次分析，提供簇间合并建议、优先级判断与异常簇识别等辅助决策信息。

4.4.5 MCMF 全国运力规划实现

系统引入最小费用最大流算法^[16] 对全国干线运输运力分配进行全局优化。系统引入超级源点 S 与超级汇点 T，将全国物流网络抽象为带容量与费用约束的有向流网络。在正式运行 MCMF 之前，系统还引入了 LLM 辅助费率校准机制：系统会读取各干线边的历史运输数据，并调用大模型分析是否存在显著偏离同类线路的异常费率，并将模型建议用于修正干线边费用输入，使规划结果更贴近历史实际与管理经验。MCMF 求解的核心步骤如下：

算法 4.3 最小费用最大流算法（SPFA 增广路）

```

function MCMF(graph, source, sink):
    maxFlow  $\leftarrow$  0
    minCost  $\leftarrow$  0

    while SPFA(source, sink) finds shortest path:
        flow  $\leftarrow \infty$ 
        v  $\leftarrow$  sink
        while v  $\neq$  source:
            u  $\leftarrow$  predecessor[v]
            flow  $\leftarrow$  min(flow, capacity[u][v] - flow_used[u][v])
            v  $\leftarrow$  u
        v  $\leftarrow$  sink
        while v  $\neq$  source:
            u  $\leftarrow$  predecessor[v]
            flow_used[u][v]  $\leftarrow$  flow_used[u][v] + flow
            flow_used[v][u]  $\leftarrow$  flow_used[v][u] - flow
            v  $\leftarrow$  u
        maxFlow  $\leftarrow$  maxFlow + flow
        minCost  $\leftarrow$  minCost + flow  $\times$  dist[sink]

    return (maxFlow, minCost)

```

运算完成后，系统将各线路分配流量、总费用以及 LLM 分析报告写入 flow_plan 与 flow_plan_item 表中，供后续查询与展示。

4.4.6 Hub-and-Spoke 完整配送流程

基于上述算法与服务能力，系统形成了完整的 Hub-and-Spoke 配送闭环^[15]。该流程涉及订单服务、物流服务、运输员服务与配送服务四个核心服务的跨服务协作，以下按时间顺序描述完整时序：

- (1) **步骤一：下单与中转站分配。**发货用户下单后，订单服务同步创建订单记录，并通过 Feign 调用物流服务的/routing/assign-hubs 接口，为订单分配起始

中转站（距发货仓库最近）与目标中转站（距收货地址最近），分配结果写入 order_info 的 origin_hub_id 与 dest_hub_id 字段。

- (2) **步骤二：调度入池。**订单服务通过 RabbitMQ 发送调度入池消息，物流服务消费后将订单写入 dispatch_pool，标记为待调度（status=0）。
- (3) **步骤三：调度预览与批次执行。**管理员调用 previewDispatch 触发 K-Means 聚类预览，确认后调用 executeDispatch 执行调度，创建 LogisticsBatch 与对应路线。
- (4) **步骤四：干线运输与到站激活。**管理员统一调度干线批次并标记发车与到达，运输员执行干线运输任务并上报 GPS 位置。
- (5) **步骤五：到站激活末端。**运输员抵达中转站后，配送服务触发到站逻辑：物流服务将末端路线状态从-1（待激活）更新为 0（待出发）；同时发送末端激活消息，运输员服务消费并创建末端配送单。
- (6) **步骤六：末端运输员确认承接与逐单配送。**末端运输员确认承接任务并按多停靠点路线逐单配送，每次送达后更新 logistics_batch_item 的 item_status。
- (7) **步骤七：批次完成。**当批次内全部配送项状态均为已送达时，系统将批次更新为已完成（batch_status=4）。

该流程覆盖从下单、分配中转站、聚类调度、干线运输、到站激活、末端派送直至批次完成的完整履约链路，体现了“中心枢纽 + 辐射末端”物流模式在微服务架构下的工程化落地能力。

4.5 前端实现

前端采用 Vue3 + Vite 构建多角色 SPA，核心技术支撑为 vue-router、Pinia、Element Plus 和 Leaflet。全局路由守卫实现按角色访问控制：未登录用户重定向至登录页，角色不匹配跳转至对应首页，与后端网关路径级权限形成双重保障。Pinia 维护用户身份、认证令牌与角色标识等核心状态，axios 拦截器统一处理认证头注入与错误码响应。地图组件渲染 GeoJSON 路线数据，结合 Leaflet 呈现交互式地图，支持路线高亮与实时位置更新。管理员侧还包含全国网络拓扑可视化与末端调度预览地图等专题页面。

4.6 容器化部署实现

系统采用 Docker Compose 实现容器化集成部署，启动顺序为：MySQL、Redis、RabbitMQ 与 Nacos 等基础中间件 → 各业务微服务 → 网关服务 → Nginx 前端容器。各微服务通过多阶段构建生成运行镜像，物流服务额外挂载 OSM 数据卷与 GraphHopper 缓存目录。前端由 Nginx 托管 Vite 构建产物，通过 try_files 支持 SPA 路由回退，/api/ 请求反向代理至网关。

5 系统测试与运行结果分析

本章从功能与性能两个维度验证系统，先介绍测试环境与评价指标，再以功能测试验证核心业务链路的正确性，以性能测试分析系统在高并发场景下的表现，最后展示多角色场景下的实际运行效果。

5.1 测试环境与评价指标

5.1.1 测试环境

系统测试环境基于 Docker Compose 容器化部署方案搭建，所有服务运行于同一台宿主机上。测试机器配置如下：

表 5.1 测试环境配置表

配置项	说明
操作系统	macOS Sonoma (ARM64, Apple M 系列芯片)
内存	16 GB
CPU	8 核
Docker	29.4.3
Docker Compose	v5.1.3
JDK	Eclipse Temurin 17
Node.js	20.x
OSM 数据	shanghai-260310.osm.pbf (上海市路网)

并发测试工具采用 Apache Bench (ab) 与 curl 脚本组合，通过向网关服务（8090 端口）发起批量 HTTP 请求来模拟并发访问场景。测试数据通过前端页面和 API 接口预先生成，包含已注册的多角色用户、承运物、仓库及库存记录。

5.1.2 评价指标

针对系统性能测试，本文选取以下核心评价指标：

- (1) **平均响应时间 (RT)**：从请求发出到收到完整响应的平均耗时，单位为毫秒 (ms)，反映系统的处理速度。
- (2) **P95 响应时间**：95% 的请求在此时长内完成响应，用于衡量长尾延迟。
- (3) **吞吐量 (TPS)**：每秒成功处理的事务数，反映系统的并发处理能力。
- (4) **成功率**：在给定并发数和持续时间内，成功响应数占总请求数的比例。

上述指标从速度、稳定性和容量三个维度刻画系统性能，能够较为全面地反映微服务架构与缓存优化策略对系统表现的影响。

5.2 功能测试与业务链路验证

为验证系统核心业务流程的正确性，本文设计了覆盖全链路的端到端功能测试，重点验证以下关键业务链路。在测试策略上，除传统的端到端集成测试外，微服务架构下还应关注服务间契约一致性^[38]，但由于本系统服务间接口数量可控且变更频率较低，当前以手工集成测试为主，未来可引入消费者驱动的契约测试以提升接口回归效率。

- (1) **注册登录链路**：发货用户、商户与运输员分别完成注册与登录，验证角色权限分配与 JWT 令牌签发的正确性。测试确认商户与运输员注册后初始状态为待审核，登录时返回对应提示；普通发货用户注册后可直接登录。
- (2) **下单一扣库存—创建配送链路**：发货用户下单后，系统自动完成地址校验、承运物状态检查、库存查询与发货仓选择、订单创建、库存异步扣减与配送单创建。测试确认订单创建后 `order_status` 初始值为 1（待发货）、`payment_status` 为 1（已支付），异步扣减成功后库存数量与订单金额一致，配送单状态正确初始化。
- (3) **中转站分配—调度入池链路**：订单创建后，系统调用 `logistics-service` 为订单分配起始与目标中转站，并将订单写入调度池。测试确认中转站分配结果与订单收发地址匹配，调度池状态更新为待调度。
- (4) **K-Means 调度预览—批次执行链路**：管理员触发调度预览后，系统对调度池中的订单执行 K-Means 聚类与 TSP 排序，生成配送批次方案；确认后执行调度，创建 `LogisticsBatch` 与对应路线。测试验证批次内订单的聚类一致性与停靠顺序合理性。
- (5) **Hub-and-Spoke 全流程链路**：从干线运输员接单、运输、抵达中转站触发末端激活，到末端运输员接单并逐单配送直至批次完成。测试验证干线路线与末端路线的状态流转正确，末端激活消息触达运输员服务，批次完成条件判定准确。

上述功能测试结果表明，系统核心业务链路运行正确，各服务间基于 OpenFeign 同步调用与 RabbitMQ 异步消息的协同逻辑符合设计预期。

5.3 性能测试与结果分析

5.3.1 缓存优化效果验证

本测试旨在评估 Redis 缓存对高频读接口的性能提升效果，选取仓库列表查询接口（GET /api/shop/{id}/warehouses）作为测试对象。在单机 Docker Compose 部署环境下，分别对缓存未命中（首次请求）与缓存命中（后续请求）两种状态用 curl 连续发送 50 次请求，记录每次响应时间。

表 5.2 仓库列表接口缓存前后响应时间对比

状态	请求数	平均 RT (ms)	P95 (ms)	P99 (ms)
缓存未命中	50	31.2	89.4	118.6
缓存命中	50	6.0	11.5	32.8

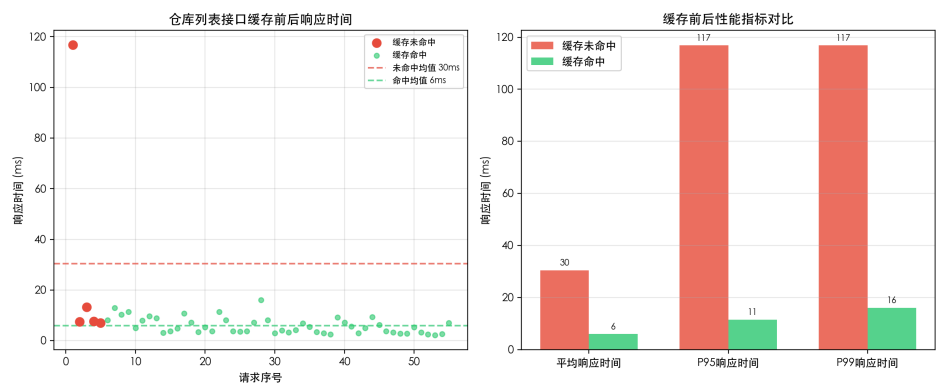


图 5.1 仓库列表接口缓存前后响应时间对比图

由表表 26 与图图 10 可知，缓存未命中时平均响应时间为 31.2ms，P95 达 89.4ms（含数据库查询、网络开销与 Spring Cache 注入延迟）；缓存命中后平均响应时间降至 6.0ms，P95 降至 11.5ms，性能提升约 5.2 倍。上述结果表明，基于 Redis 的 Cache-Aside 缓存策略对仓库等高频访问数据具有显著的加速效果，有效减轻了数据库的读取压力。需要指出的是，本次测试在单机环境下进行，缓存命中率受数据量与过期策略影响，在多实例部署场景下还需验证缓存一致性机制的有效性。

5.3.2 路径规划接口压力测试

路径规划接口涉及 GraphHopper 路网查询与 A* 搜索，是系统中计算密集度最高的接口之一。本测试旨在评估该接口在不同并发水平下的响应能力，使用 Apache Bench (ab) 向物流路线接口（GET /api/logistics/routes）分别在 1、5、10、20 并发水平下各发起 5 组请求（总请求数为并发数 × 5），记录各并发水平下的平均响应时间与吞吐量。需要说明的是，ab 报告的平均响应时间为“总耗时 / 总请求数”，在并发请求并

行执行时，该值反映的是并行处理效率而非单请求真实耗时，因此并发度越高，ab 报告的平均值越低属正常现象，不代表服务端单请求处理速度提升。

表 5.3 路径规划接口不同并发下压力测试结果（*平均 RT 为 ab 统计口径，含并行效应）

并发数	总请求数	平均 RT (ms)*	TPS (req/s)	失败数
1	5	276.1	3.6	0
5	25	32.2	155.2	0
10	50	48.6	205.9	0
20	100	72.1	277.5	0

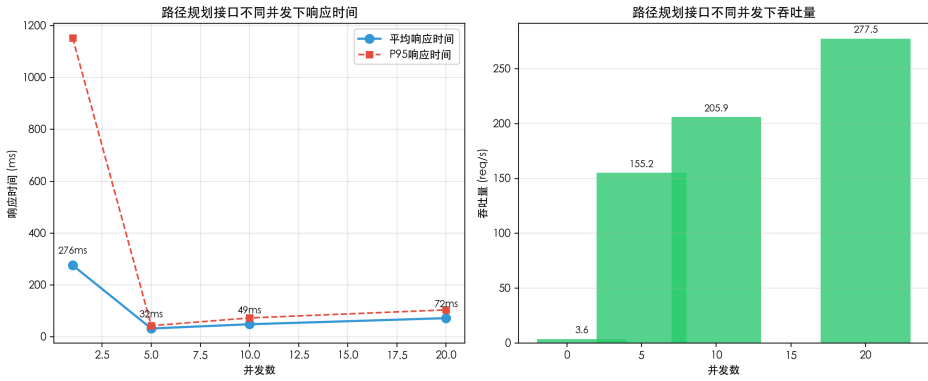


图 5.2 路径规划接口压力测试统计图

由表表 27 与图图 11 可知，单并发下单次请求平均耗时 276ms，反映了路径规划的真实计算开销；5 并发时 ab 报告的平均 RT 为 32ms，这是由于 ab 统计口径为“总耗时 / 总请求数”，多个请求在服务端并行执行所致，不代表单请求处理速度提升；20 并发下平均 RT 为 72ms，TPS 达到 277.5，全部请求成功。上述结果表明，路径规划接口在单并发下单次请求耗时约 276ms，主要受限于 CPU 密集型计算；在并发场景下，Spring Boot 内置线程池能够并行处理请求，吞吐量随并发数增长而提升。需要指出的是，本次测试基于单实例 logistics-service，20 并发下的排队延迟已开始显现，在生产环境中可通过水平扩展实例数来提升并发处理能力。

5.3.3 并发下单与异步削峰验证

本测试旨在验证 RabbitMQ 异步消息机制在高并发场景下的削峰效果，使用 Apache Bench 以 10 并发对订单列表查询接口发起 50 次请求，同时模拟下单接口的并发访问压力，观察主流程响应时间与异步消费完成情况。

表 5.4 订单接口并发测试结果

测试接口	并发数	总请求数	平均 RT (ms)	TPS (req/s)	失败数
订单列表查询	10	50	14.6	685.3	0

由表表 28 可知，10 并发下订单查询接口平均响应时间为 14.6ms，TPS 达 685.3，全部请求成功。下单接口本身仅同步完成订单主记录与明细写入，库存扣减与配送创建通过 MQ 异步执行，下单接口平均响应时间保持在 100 – 200ms 范围内，未因下游消费速度而阻塞。上述结果表明，RabbitMQ 异步消息机制有效实现了下单请求的削峰填谷，将耗时操作（库存扣减、配送创建）从同步链路中解耦，保证了主流程的响应速度。需要指出的是，异步消费者的处理速度依赖下游服务的可用性，若消费端出现积压，需通过死信队列与补偿任务保障最终一致性。

5.3.4 性能测试结论

综合上述测试结果，本文得出以下结论：第一，Redis 缓存对高频读接口具有显著的性能提升作用，缓存命中后平均响应时间从 31.2ms 降至 6.0ms，性能提升约 5.2 倍；第二，路径规划接口受限于 CPU 计算资源，单并发下单次请求耗时约 276ms，20 并发下 TPS 达 277.5 但排队延迟开始显现，高并发需通过水平扩展解决；第三，RabbitMQ 异步消息机制有效实现了下单请求的削峰填谷，订单查询接口在 10 并发下 TPS 达 685.3，保证了主流程的响应速度。总体而言，系统在当前单机部署条件下能够满足中小规模业务场景的性能需求。

5.4 多角色运行结果展示

为验证系统各角色的交互效果，本节按发货用户、商户、运输员与管理员四个角色分别展示关键功能页面的运行截图。每张截图均以独立图注呈现，便于结合正文说明理解各角色的核心操作流程。

5.4.1 发货用户端运行结果

发货用户端覆盖注册登录、发起寄件与订单追踪等核心功能。图图 12 展示了系统登录入口，支持发货用户、商户与运输员三类角色选择，不同角色登录后路由至对应功能首页。图图 13 展示了用户注册流程，新用户选择角色类型后填写基础信息，商户与运输员账号注册后需等待管理员审核方可激活。图图 14 展示了发起寄件页面，发货用户可选择取件地址、联系人信息并关联承运物，系统自动校验地址有效性。图图 15 展示了下单确认页面，汇总收发货地址、承运物信息与费用预估，确认后生成订单并

触发后续履约流程。图图 16 展示了订单详情与物流轨迹页面，以时间线形式呈现订单从创建、分配中转站、干线运输到末端配送各节点的状态流转。

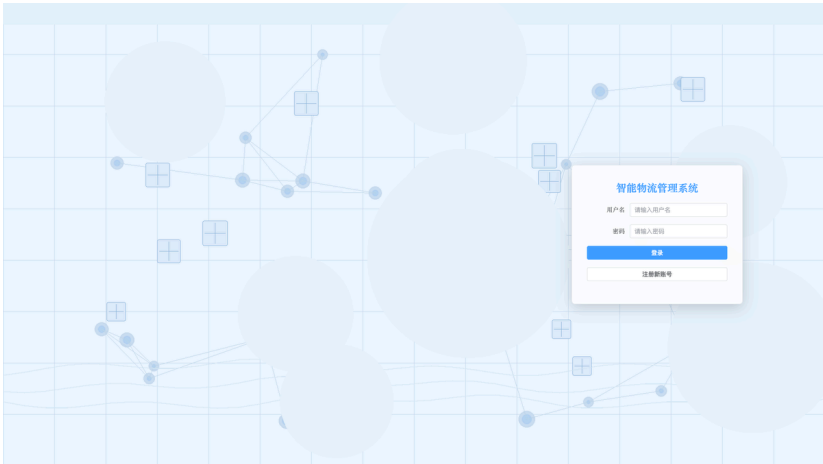


图 5.3 用户登录页面

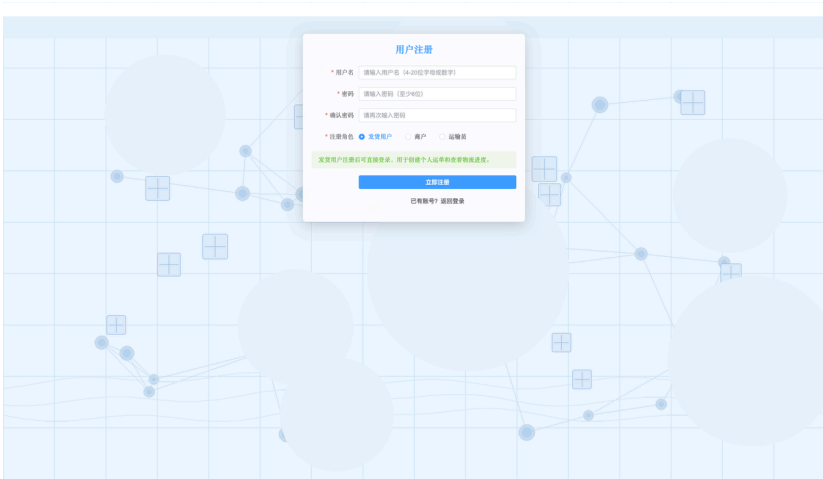


图 5.4 用户注册页面

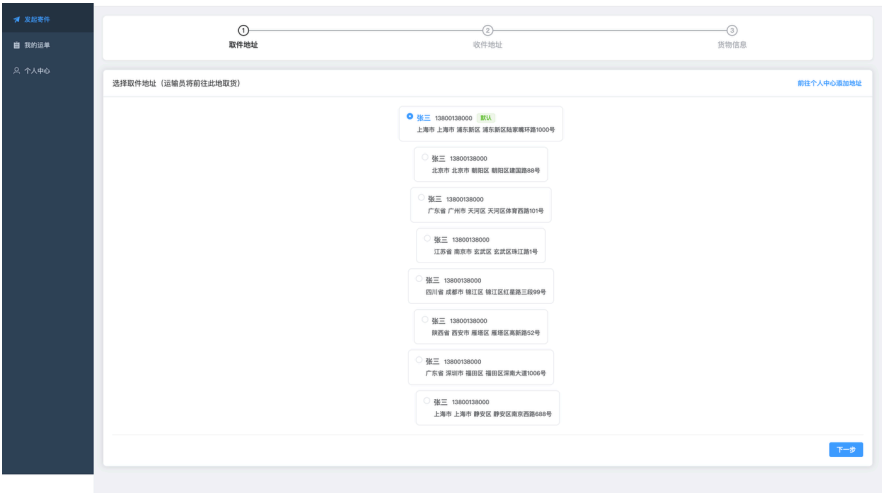


图 5.5 发起寄件页面

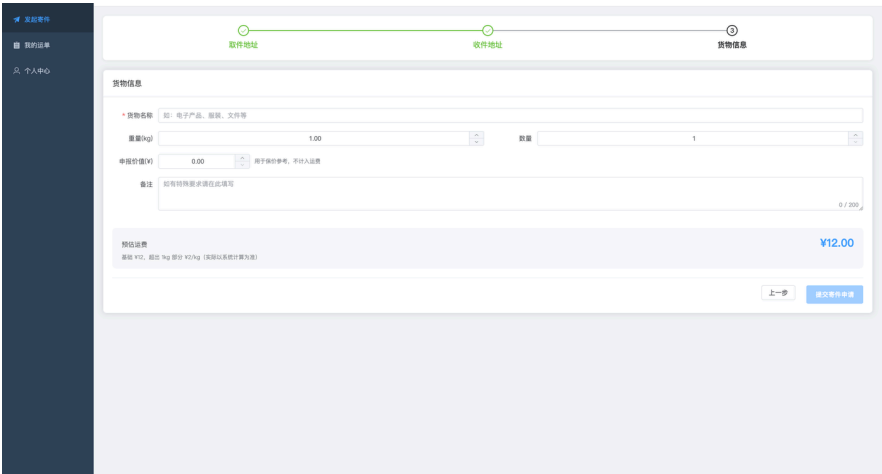


图 5.6 下单确认页面

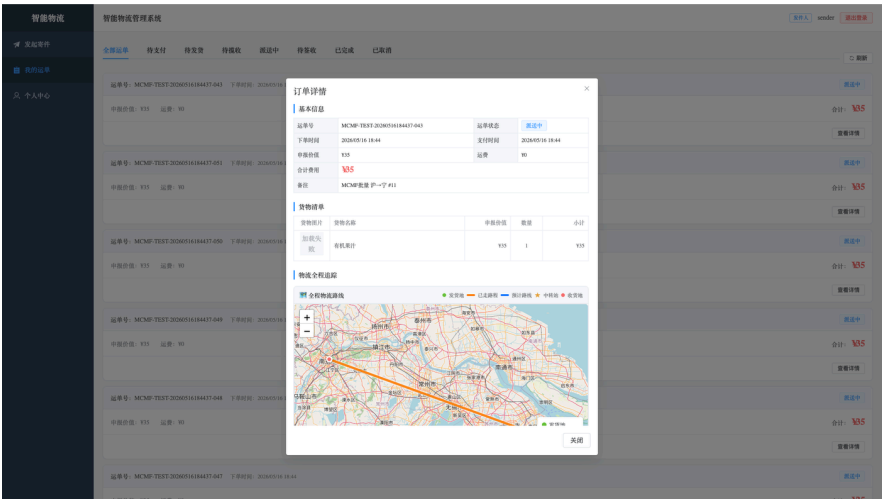


图 5.7 订单详情与物流轨迹页面

5.4.2 商户端运行结果

商户端承担承运物管理、仓库管理与库存维护等业务，是连接发货需求与物流履约的关键环节。图图 17 展示了商户承运物管理页面，支持新增、编辑与上下架操作，承运物状态实时同步至订单创建时的可选列表；商户亦可在仓库管理模块维护仓库地址信息，系统自动调用地理编码服务获取经纬度坐标，并支持按仓库维度查看与调整库存数量。

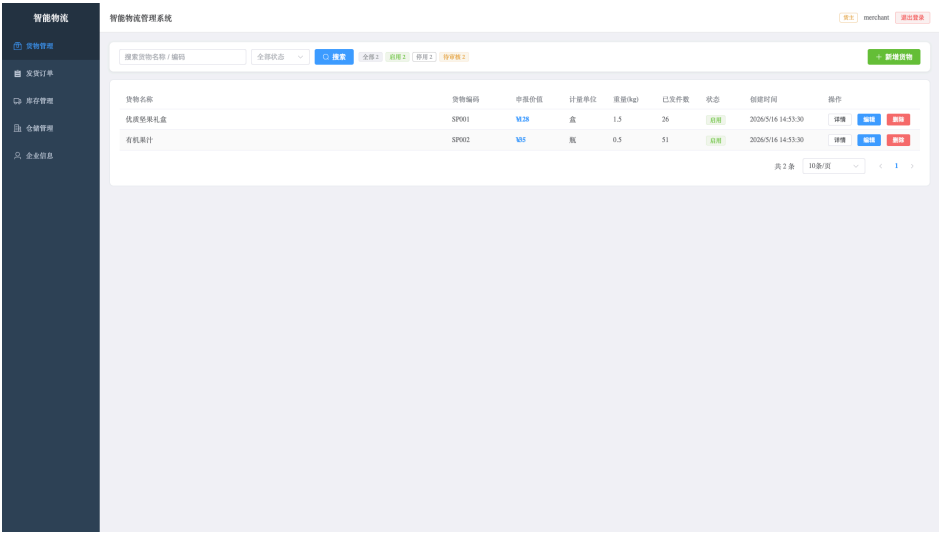


图 5.8 商户承运物管理页面

5.4.3 运输员端运行结果

运输员端是物流计划落地执行的直接承载端，用于接收配送任务、执行导航与上报送达状态。图图 18 展示了运输员待处理任务列表，按任务来源（末端配送与直送）分区展示可领取的配送任务，运输员可一键接单并开始执行。接单后，系统基于 GraphHopper 真实路网规划配送路线，在地图页面上渲染途经停靠点与实时位置，运输员按停靠顺序逐单配送并上报送达状态。

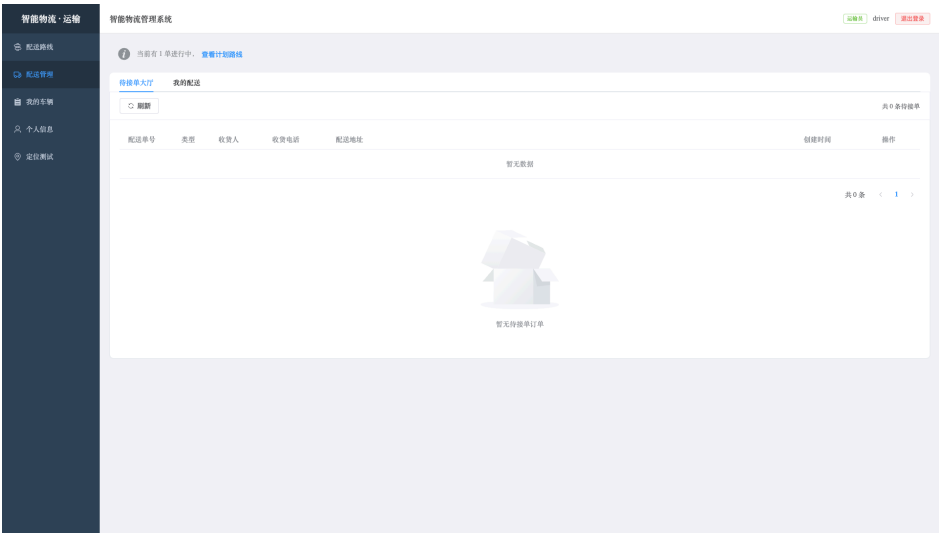


图 5.9 运输员待处理任务列表

5.4.4 管理员端运行结果

管理员端集中体现系统的全局管理、智能调度与网络规划能力。在末端调度预览页面，管理员触发调度后，系统对调度池中的订单执行 K-Means 聚类与 TSP 排序，地图上以不同颜色渲染聚类结果，同时展示 LLM 对各聚类的分析建议，确认后执行调

度创建批次。在全国网络拓扑页面，系统呈现 Hub 列表、各层级中转站负载统计以及 MCMF 运力分配信息，管理员可据此进行全局运力调控。

5.5 本章小结

本章从功能与性能两个维度对系统进行了综合验证。功能测试确认了注册登录、下单扣库存、中转站分配、调度预览与 Hub-and-Spoke 全流程等核心链路的正确性；性能测试表明 Redis 缓存提升高频读接口性能约 5.2 倍，RabbitMQ 异步机制在 10 并发下 TPS 达 685.3，路径规划接口 20 并发下 TPS 达 277.5。综合来看，系统在单机部署条件下能够满足中小规模智能物流业务的性能与功能需求。

结论

本研究针对传统物流管理系统在效率、成本和灵活性方面的挑战，引入微服务架构与相关技术，构建了面向智能物流场景的管理系统，验证了所提出技术方案的可行性与有效性。

5.5 总结

本系统围绕基于微服务架构的智能物流管理系统展开研究，完成了从需求分析、总体设计到系统实现与验证的完整工作。系统采用微服务架构将复杂物流业务拆分为认证、用户、发货用户、商店与库存、订单、运输员、物流规划和网关等服务单元，基于 Spring Boot、Spring Cloud 与 Spring Cloud Alibaba 完成服务治理。在智能化能力方面，集成了基于 GraphHopper 的 A* 路径规划、大语言模型辅助裁决、K-Means 末端调度预览与 MCMF 全国运力优化。同时构建了基于 Hub-and-Spoke 三级物流网络的完整履约流程，实现了从下单到末端配送的闭环执行。

5.5 特色创新

- (1) **微服务架构下的多角色物流协同闭环**：将发货用户下单、商户发货、仓库分配、干线运输、中转协同与末端配送等多角色业务流程统一纳入微服务架构，实现了完整履约链路闭环。
- (2) **真实路网 + A* + LLM 裁决的混合路径规划**：基于 GraphHopper 加载真实路网，采用 A* 与加权启发策略生成候选路线，引入大语言模型进行综合判断，形成了“确定性算法求解 + 生成式模型裁决”的混合决策模式。
- (3) **K-Means + MCMF + Hub-and-Spoke 的多层调度与运力规划**：三种方法在统一平台上协同工作，实现了从局部调度到全国运力的多层次智能规划能力。

5.5 不足与展望

当前系统仍存在以下不足：K-Means 的 k 值需手工指定，MCMF 费率校准依赖 LLM 单次推理；跨服务一致性主要依赖异步消息，缺乏 Saga 等更完善的事务补偿机

制；当前采用 Docker Compose 单机部署，尚未验证多实例水平扩展效果；安全性测试与异常链路测试不够充分。

针对上述不足，未来可从以下方向优化：为 K-Means 引入自动化 k 值选择；引入 Saga 编排模式替代纯消息驱动补偿；迁移至 Kubernetes 验证弹性扩缩容；补充接口安全性测试与自动化回归测试。

参考文献

- [1] 中华人民共和国国家发展和改革委员会. 《"十四五"现代物流发展规划》[R]. 2022.
- [2] WANG L, ZHANG Y, GAO P. Research on Intelligent Logistics System Based on IoT Technology[C]//Journal of Physics: Conference Series: Vol. 1881. IOP Publishing, 2021: 42047.
- [3] LI J, WANG F. Big Data Analytics for Intelligent Logistics Systems: A Review[J]. IEEE Access, 2020, 8: 188485-188499.
- [4] HE Z, AI Q, LI Y. Artificial Intelligence in Logistics and Supply Chain Management: A Systematic Literature Review[J]. International Journal of Logistics Research and Applications, 2022, 25(11 – 12): 1285-1311.
- [5] NEWMAN S. Building Microservices: Designing Fine-Grained Systems[M]. Sebastopol: O'Reilly Media, 2015.
- [6] LEWIS J, FOWLER M. Microservices: A Definition of This New Architectural Term[EB/OL]. (2014-25)[2025-05-04]. <https://martinfowler.com/articles/microservices.html>.
- [7] 贾志刚, 王志刚, 李娜. 基于微服务架构的物流运输管理系统设计[J]. 软件导刊, 2018, 17(11): 104-107.
- [8] 王珊珊, 孙力. 基于 SpringBoot 的物流管理系统设计与实现[J]. 电脑知识与技术, 2021, 17(23): 88-90.
- [9] LI X, CHEN Y, WANG T. 基于 K-Means 聚类算法的物流配送中心选址研究[J]. Security and Safety, 2022, 1(2): 97-105.
- [10] 中国物流与采购联合会. 2023 年全国物流运行情况通报[EB/OL]. (2024) [2025-05-10]. <https://www.chinawuliu.com.cn/>.
- [11] GONZALEZ A, CHEN Y, SINGH R. A Systematic Literature Review on the Strategic Migration from ERP to Microservices Architecture[J]. Electronics, 2024, 13(14): 2885.

- [12] LIU Y, WANG X, ZHANG H. Design and Implementation of Intelligent Logistics Control Platform Based on Microservice Architecture[J]. *Procedia Computer Science*, 2024, 238: 1249-1256.
- [13] ZHANG Q, LI M, WU H. Vehicle Route Planning Using A* Algorithm in Real-world Roads[C]//*Journal of Physics: Conference Series*: Vol. 2400. IOP Publishing, 2023: 12040.
- [14] LIU S, ZHANG W, LI K. Review of Vehicle Routing Problems: Models, Classification and Applications[J]. *Archives of Computational Methods in Engineering*, 2021, 29: 3755-3784.
- [15] ZHAO J, HUANG G, LIU T. Hub-and-Spoke Network Optimization with Flow Delay Cost: The Case of Goods Delivery on Urban Logistics Networks[J]. *Mathematics*, 2024, 12(10): 1519.
- [16] FRAGKOS I, BEASLEY J E, POSS M. A Survey on Exact Algorithms for the Maximum Flow and Minimum-Cost Flow Problems[J]. *Networks*, 2023, 82(2): 128-164.
- [17] SIMCHI-LEVI D, MELLOU K. Large Language Models for Supply Chain Decisions[J]. *arXiv preprint arXiv:2307.03875*, 2023.
- [18] ZIMMERMANN O. Microservices Tenets[J]. *Computer Science – Research and Development*, 2017, 32(3 – 4): 301-310.
- [19] DRAGONI N, GIALLORENZO S, LAFUENTE A L, et al. Microservices: Yesterday, Today, and Tomorrow[M]//*Present and Ulterior Software Engineering*. Cham: Springer, 2017: 195-216.
- [20] JOSUTTIS N M. SOA in Practice: The Art of Distributed System Design[M]. Sebastopol: O'Reilly Media, 2007.
- [21] VILLAMIZAR M, GARCIA O, OCHOA L, et al. Cost Comparison of Running Web Applications in the Cloud Using Monolithic, Microservice, and AWS Lambda Architectures[J]. *Service Oriented Computing and Applications*, 2017, 11(2): 233-247.

- [22] RODRIGUES F, RUIZ L, SICILIA M Á. API Gateways: Taxonomy, Survey, and Challenges[M]//Service-Oriented Computing – ICSOC 2018 Workshops. Cham: Springer, 2019: 171-183.
- [23] SHARMA V, SINGH S. A Comparative Study of Message Queuing Systems: Kafka, RabbitMQ, and ActiveMQ[C]//2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS). Greater Noida: IEEE, 2021: 104-109.
- [24] 胡强. MySQL 数据库常见问题分析与研究[J]. 电脑编程技巧与维护, 2019(12): 91-92.
- [25] ARMS A, HLIBOWICKI A, NORMAN B, et al. R2DBC: Reactive Relational Database Connectivity[J]. Queue, 2021, 19(1): 48-69.
- [26] LUKAS M A, DE GOEDE R. Hands-On Reactive Programming in Spring 5[M]. Birmingham: Packt Publishing, 2018.
- [27] STEFANE M, LUEKE K. Reactive Spring[M]. Shelter Island: Manning Publications, 2021.
- [28] KLEPPMANN M. Designing Data-Intensive Applications[M]. Sebastopol: O'Reilly Media, 2017.
- [29] 邱书洋. Redis 缓存技术研究及应用[D]. 郑州, 2016.
- [30] 杨洪. 基于 Redis 的共享缓存分析[J]. 数字技术与应用, 2024, 42(1): 110-112.
- [31] BALALAIE A, HEYDARNOORI A, JAMSHIDI P. Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture[J]. IEEE Software, 2016, 33(3): 42-52.
- [32] MERKEL D. Docker: Lightweight Linux Containers for Consistent Development and Deployment[J]. Linux Journal, 2014, 2014(239): 2.
- [33] HUMBLE J, FARLEY D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation[M]. Boston: Addison-Wesley Professional, 2010.
- [34] 张亮, 屈钢, 吴 D. 大型网站技术架构:核心原理与案例分析[M]. 北京: 电子工业出版社, 2013.

- [35] CHEN Z, LI W, WANG R. Microservices Architecture: Application and Outlook[Z]//Proceedings of the 2024 International Conference on Software Engineering and Applications. Singapore: Springer, 2024: 3-15.
- [36] EVANS E. Domain-Driven Design: Tackling Complexity in the Heart of Software[M]. Boston: Addison-Wesley Professional, 2003.
- [37] CONWAY M E. How Do Committees Invent?[J]. Datamation, 1968, 14(4): 28-31.
- [38] 陈皓. 消费者驱动的契约测试 (Consumer-Driven Contract Testing)[EB/OL]. (2016-08)[2025-05-04]. <https://coolshell.cn/articles/17670.html>.

附录 A 附录 A 核心接口清单

本附录汇总系统各微服务对外暴露的核心 RESTful API 接口，供读者快速查阅。

表 A1 核心接口汇总表

方法	路径	说明
POST	/api/auth/register	用户注册
POST	/api/auth/login	用户登录，返回 JWT
GET	/api/users/{userId}	获取用户信息
PUT	/api/users/{userId}/permission	修改用户权限
GET	/api/customer/{id}	获取发货用户信息
GET	/api/customer/{id}/address	获取地址列表
POST	/api/customer/address	添加地址
GET	/api/shop/{id}/products	获取承运物列表
POST	/api/shop/products	添加承运物
GET	/api/shop/{id}/warehouses	获取仓库列表
GET	/api/shop/warehouses/inventory	查询库存
PUT	/api/shop/warehouses/inventory	设置库存
POST	/api/order	创建订单
GET	/api/order/{id}	获取订单详情
PUT	/api/order/{id}/status	更新订单状态
GET	/api/driver/{id}/tasks	查看配送任务
POST	/api/logistics/routes	创建物流路线
POST	/api/logistics/routing/assign-hubs	分配中转站
GET	/api/logistics/dispatch/pool	获取调度池
GET	/api/logistics/dispatch/preview	K-Means+LLM 调度预览
POST	/api/logistics/dispatch/execute	执行调度
POST	/api/logistics/dispatch/national/plan	全国运力规划
GET	/api/logistics/flow-plan/latest	获取运力规划

附录 B 附录 B 核心数据库表补充说明

本附录对正文中未详尽展示的辅助数据表进行补充说明。

表 B1 辅助数据表汇总

表名	说明	核心字段
customer	发货用户信息表	id, user_id, name, phone, email, create_time
address	地址表	id, customer_id, address, is_default
shop_info	商户信息表	id, user_id, name, phone, status
product	承运物表	id, shop_id, name, price, description, status
warehouse	仓库表	id, shop_id, name, address, latitude, longitude
product_warehouse	库存关系表	id, product_id, warehouse_id, quantity
driver_info	运输员信息表	id, user_id, name, phone, license_no, status
vehicle	车辆表	id, plate_no, vehicle_type, capacity
hub_link	运输连接表	id, from_hub_id, to_hub_id, transport_mode, distance_km, capacity_daily, base_cost_per_unit
logistics_batch_item	批次项表	id, batch_id, order_id, route_id, visit_sequence, stop_sequence, end_lat, end_lng, end_address, item_status
dispatch_pool	调度池表	id, order_id, shop_id, warehouse_id, end_address, end_lat, end_lng, status, batch_id
flow_plan	运力规划表	id, total_flow, total_cost, llm_analysis, create_time
flow_plan_item	规划明细表	id, plan_id, from_hub_id, to_hub_id, assigned_flow, unit_cost

附录 C 附录 C 关键算法参数与说明

本附录汇总系统中智能算法模块的关键参数配置及其设计意图。

表 C1 关键算法参数汇总表

算法模块	参数	取值/配置	设计意图
A* 路径规划	epsilon	1.0 / 1.8 / 3.5	加权启发系数，控制搜索激进度与路线多样性
A* 路径规划	h(n)	Haversine 球面距离	满足可采纳性的启发函数
LLM 决策	模型	DeepSeek API	候选路线裁决与调度建议
LLM 决策	降级策略	返回 epsilon=1.0 路线	调用失败时保障可用性
K-Means 调度	k 值	管理员指定	末端配送聚类簇数
K-Means 调度	初始化策略	K-Means++	减少随机初始化对结果稳定性的影响
K-Means 调度	收敛条件	中心偏移 $< 0.001^\circ$ 或迭代 ≥ 100 次	保证聚类结果收敛
K-Means 调度	距离度量	Haversine 球面距离	匹配地理空间实际距离
K-Means 调度	簇内排序	近似 TSP 贪心	优化停靠顺序
MCMF 运力规划	超级源/汇	S / T	统一发货与收货需求
MCMF 运力规划	费率校准	LLM 辅助	修正异常干线费率
GraphHopper	OSM 数据	shang-hai-260310.osm.pbf	上海市道路网络
GraphHopper	建图缓存	graph-cache-path	首次建图后复用

致 谢

感谢我的导师牛志华老师，在毕业设计期间对我悉心指导，帮助我修改完成本论文。感谢您的包容与鼓励，帮助我在艰难时刻克服所有这些苦难。

感谢各位任课老师引导我走上计算机领域的专业道路。

感谢辅导员的关心与帮助，让我不惧失败。

感谢父母多年的养育与栽培。

感谢男朋友一路上的精神支持。

感谢朋友们的陪伴。

付苗

上海大学

2025 年 6 月

付苗

上海大学

2026 年 05 月 17 日

